

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH

ĐỒ ÁN MÔN HỌC

< Thiết kế và tối ưu hóa lõi IP CORDIC 32-bit trên FPGA >

< GVHD: ThS. Ngô Hiếu Trường >

< Sinh viên thực hiện >

Trần Danh Vinh 23521798

Huỳnh Tấn Vũ 23521806

TP. HỒ CHÍ MINH, 2026

MỤC LỤC

TỔNG QUAN	1
1.1. Đặt vấn đề	2
1.2. Mục tiêu đề tài	3
1.3. Đối tượng và phạm vi nghiên cứu	3
1.3.1. Đối tượng nghiên cứu	3
1.3.2. Phạm vi nghiên cứu và giới hạn đề tài	3
Chương 2. CƠ SỞ LÝ THUYẾT	4
2.1. FPGA	4
2.1.1. LEs (Logic Elements)	4
2.1.2. Registers	4
2.2. Thuật toán CORDIC	5
2.2.1. Lịch sử	5
2.2.2. Nguyên lý toán học cốt lõi	5
2.2.2.1. Phép xoay tọa độ cơ bản trên mặt phẳng.	5
2.2.3. Chế độ xoay góc (Rotation mode)	7
2.2.4. Chế độ tìm góc (Vectoring mode)	8
2.3. Chuẩn biểu diễn số phẩy tĩnh (Fixed-Point)	9
2.3.1. Sự cần thiết cho kiến trúc FPGA	9
2.3.2. Định dạng Q2.29 cho trục tọa độ	9
2.3.3. Ánh xạ cho trục góc Z	10
Chương 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	11
3.1. Sơ đồ khối tổng quát	11
3.2. Thiết kế khối tiền xử lý (Preprocessor)	12
3.2.1. Chức năng	12
3.2.2. Nguyên lý hoạt động	13
3.3. Thiết kế lõi CORDIC	15
3.3.1. Bảng tra hằng số góc (LUT)	15
3.3.2. Khối DATAPATH.	16
3.3.2.1. Chức năng của khối DATAPATH.	17
3.3.2.2. Nguyên lý hoạt động.	17
3.3.3. Khối CORDIC_CORE	18
3.3.3.1. Chức năng của khối CORDIC_CORE.	18
3.3.3.2. Nguyên lý hoạt động của CORDIC_CORE	19
3.4. Thiết kế khối hậu xử lý (Postprocessor)	20
3.4.1. Chức năng	20

3.4.2. Nguyên lí hoạt động	21
Chương 4. MÔ PHỎNG VÀ ĐÁNH GIÁ HIỆU NĂNG	22
4.1. Xây dựng testbench kiểm chứng	22
4.2. Đánh giá kết quả mô phỏng chức năng (Pre-simulation)	24
4.3. Đánh giá kết quả mô phỏng mức cổng vật lý (Post-simulation)	49
4.3.1. Mục đích và môi trường kiểm chứng	49
4.3.2. Đánh giá độ trễ đường ống	49
4.3.3. Đánh giá kết quả cho các testcases	51
4.4. Đánh giá thông số phần cứng	54
4.4.1. Tài nguyên sử dụng (Logic Elements, Registers)	54
4.4.2. Tần số hoạt động tối đa (Fmax) và Độ trễ (Latency)	55
4.4.2.1. Tần số hoạt động tối đa (Fmax)	55
4.4.2.2. Độ trễ (Lantency)	55
4.4.3. Phân tích sai số	55
Chương 5. KẾT LUẬN	57
5.1. Kết quả đạt được	57
5.2. Những điểm hạn chế của đề tài	57

TÓM TẮT ĐỒ ÁN

Trong lĩnh vực viễn thông số, xử lý tín hiệu radar và điều khiển hệ thống nhúng, việc tính toán các hàm lượng giác (Cos, Sin) và tính toán vector đòi hỏi tốc độ đáp ứng cực kỳ khắt khe. Nếu thực hiện các phép toán này bằng vi xử lý mềm thông thường, hệ thống sẽ gặp phải giới hạn lớn về độ trễ và thông lượng. Do đó, việc chuyển đổi thuật toán xuống chạy trực tiếp trên lớp vật lý là một giải pháp thiết yếu.

Mục tiêu cốt lõi của đồ án này là nghiên cứu, thiết kế và hiện thực hóa thành công lõi IP CORDIC (COordinate Rotation DIgital Computer) bằng ngôn ngữ mô tả phần cứng Verilog HDL. Lõi CORDIC được thiết kế với độ rộng dữ liệu 32-bit với chuẩn số phẩy tĩnh Fixed-point và cấu trúc đường ống pipelining 20 tầng, cho phép hệ thống tính toán đồng thời các giá trị lượng giác và độ lớn vector với độ chính xác cao.

Qua thời gian thực hiện đề tài, nhóm đã thiết kế được cấu trúc tối ưu, xây dựng hoàn chỉnh các khối Tiền xử lý (Preprocessor) để bao quát toàn bộ 4 góc phần tư 360 độ, lõi tính toán (CORDIC Core) và Hậu xử lý (Postprocessor). Tích hợp thành công các khối nhân cứng (DSP Blocks) trên FPGA để xử lý hệ số bù Gain, giúp giảm tải cho các cổng logic tổ hợp thông thường. Thông qua kỹ thuật Pipelining và tinh chỉnh ràng buộc định thời, thiết kế đã khắc phục được đường găng (Critical Path) để đạt tần số hoạt động tối đa lên đến 128.12 MHz, với độ trễ (Latency) là 24 chu kỳ. Xây dựng môi trường Testbench tự động, mô phỏng mức công vật lý (Post-simulation) trên ModelSim.

TỔNG QUAN

1.1. Đặt vấn đề

Trong sự phát triển bùng nổ của các hệ thống điện tử hiện đại, đặc biệt là trong lĩnh vực viễn thông số, xử lý tín hiệu radar, robot học và điều khiển động cơ (FOC), việc tính toán các hàm lượng giác (Sin, Cos) và các phép biến đổi vector đóng một vai trò vô cùng thiết yếu. Các hệ thống này thường yêu cầu tính toán liên tục với độ chính xác cao và thời gian đáp ứng cực kỳ khắt khe (chuẩn thời gian thực - real-time).

Theo phương pháp truyền thống, trên các hệ thống vi điều khiển hoặc vi xử lý thông thường, các phép toán lượng giác được thực hiện bằng phần mềm thông qua các thư viện toán học. Tuy nhiên, phương pháp này đòi hỏi thực hiện các phép nhân phức tạp, tiêu tốn từ hàng chục đến hàng trăm chu kỳ máy cho một kết quả, tạo ra hiện tượng nút thắt cổ chai về mặt tốc độ. Một giải pháp khác là sử dụng bảng tra (Lookup Table - LUT) để lưu sẵn kết quả, nhưng nếu yêu cầu độ phân giải lớn như chuẩn 32-bit, phương pháp này sẽ làm cạn kiệt dung lượng bộ nhớ RAM/ROM của hệ thống.

Để giải quyết bài toán trên, thuật toán CORDIC (COordinate Rotation DIgital Computer) được biết đến như một giải pháp phần cứng kinh điển và tối ưu bậc nhất. Thuật toán này biến đổi các phép toán lượng giác phức tạp thành một chuỗi các phép dịch bit (Shift) và phép cộng/trừ (Add/Subtract) đơn giản. Khi được hiện thực hóa trên các chip logic lập trình được như FPGA, kết hợp cùng kiến trúc xử lý song song và kỹ thuật đường ống Pipelining, CORDIC có thể tận dụng triệt để sức mạnh vật lý để tính toán lượng giác.

Nhận thấy được tầm quan trọng, tính ứng dụng thực tiễn cao của thuật toán CORDIC, cũng như mong muốn làm chủ quy trình thiết kế vi mạch số từ khâu viết mã RTL, tối ưu hóa định thời đến mô phỏng mức cổng Gate-level, nhóm đã quyết

định chọn đề tài: "**Thiết kế và tối ưu hóa lõi IP CORDIC 32-bit trên FPGA**" làm đề án nghiên cứu.

1.2. Mục tiêu đề tài

Nghiên cứu, thiết kế và tối ưu hóa thành công một lõi thực thi thuật toán CORDIC trên nền tảng công nghệ FPGA. Lõi phần cứng được thiết kế bằng ngôn ngữ mô tả phần cứng Verilog HDL, chỉ sử dụng các bộ cộng/ trừ (Add/Subtract) và dịch bit đơn giản không sử dụng bộ nhân nhằm tiết kiệm tối đa tài nguyên khả trình, đồng thời áp dụng các kỹ thuật kiến trúc máy tính để đạt được tần số hoạt động cao > 100 MHz, đáp ứng nhu cầu tính toán trong các hệ thống thời gian thực.

1.3. Đối tượng và phạm vi nghiên cứu

1.3.1. Đối tượng nghiên cứu

- Thuật toán máy tính số xoay tọa độ CORDIC bao gồm chế độ xoay góc (Rotation) và chế độ tìm vector (Vectoring).
- Kỹ thuật thiết kế vi mạch số và tối ưu hóa đường ống Pipelining.
- Ngôn ngữ mô tả phần cứng Verilog HDL.

1.3.2. Phạm vi nghiên cứu và giới hạn đề tài

- Hệ thống được thiết kế và cố định ở độ rộng dữ liệu 32 bit và giới hạn ở 20 vòng lặp.
- Đồ án tập trung vào việc hiện thực hóa và kiểm chứng thành công ở mức vi kiến trúc (RTL Simulation) và mức cổng vật lý (Gate-level Post-simulation) trên phần mềm ModelSim/Quartus.

Chương 2. CƠ SỞ LÝ THUYẾT

2.1. FPGA

FPGA (Field-Programmable Gate Array) là mảng cổng logic có thể lập trình được, cho phép các kỹ sư vi mạch thiết kế và cấu trúc lại lớp vật lý của phần cứng theo những yêu cầu chuyên biệt. Khác với các hệ thống vi xử lý chạy lệnh tuần tự, sức mạnh của FPGA nằm ở khả năng xử lý song song với độ trễ tính bằng nano-giây. Để hiện thực hóa lõi IP CORDIC, đồ án tập trung khai thác và quản lý nguồn tài nguyên cốt lõi trên cấu trúc chip:

2.1.1. LEs (Logic Elements)

- Phần tử logic (LE) là khối xây dựng cơ bản và nhỏ nhất trên kiến trúc các dòng chip FPGA, thường bao gồm các thành phần chính: bảng tra LUT, chuỗi cờ nhớ (Carry chain), bộ dồn kênh (Multiplexer).
- Trong thiết kế CORDIC 32 bit, mạng lưới hàng nghìn LEs được sử dụng để xây dựng nên các khối Datapath (bộ cộng, bộ trừ Ripple Carry Adder), bộ dịch bit và các mạch tổ hợp khôi phục tọa độ cho khối Tiền xử lý (Preprocessor) và Hậu xử lý (Postprocessor).

2.1.2. Registers

- Nằm phía sau LUT bên trong mỗi LE chính là D-Flipflop hoặc thanh ghi. Đặc tính của D-FF là chỉ cập nhật và chốt dữ liệu từ ngõ vào D ra ngõ ra Q tại thời điểm xảy ra sườn xung nhịp lên - posedge của clock.
- Việc chèn các thanh ghi này vào giữa các mạng lưới LEs chính là cốt lõi của kỹ thuật Pipelining. Khi thiết kế lõi CORDIC 20 tầng lặp và sử dụng chuỗi các bộ cộng/trừ, dịch bit ở khối Hậu xử lý thay cho bộ nhân, đồ án đã dùng thanh ghi để chốt dữ liệu qua các tầng. Tín hiệu thay vì phải chạy xuyên qua hàng trăm cổng logic trong một chu kỳ thì chỉ cần chạy qua một đoạn ngắn rồi chốt lại. Điều này giúp cắt đứt đường găng (Critical Path), giúp tăng tần số hoạt động $F_{\max}$ của toàn hệ thống.

2.2. Thuật toán CORDIC

2.2.1. Lịch sử

- Vào những năm 1950, phần cứng máy tính rất hạn chế, các bộ nhân làm tốn quá nhiều tài nguyên và chạy chậm. Thuật toán CORDIC được Jack E. Volder phát minh vào năm 1959 như một giải thuật đột phá để tính toán lượng giác chỉ bằng các phép toán cơ bản nhất: Dịch bit(Shift) và Cộng/Trừ (Add/Sub).

2.2.2. Nguyên lý toán học cốt lõi

2.2.2.1. Phép xoay tọa độ cơ bản trên mặt phẳng.

- Giả sử ta có một vector với tọa độ ban đầu là (X, Y) , khi thực hiện xoay vector này đi một góc θ quanh gốc tọa độ để tạo ra vector mới (X', Y') , ta có hệ phương trình.

$$X' = X \cos(\theta) - Y \sin(\theta)$$

$$Y' = X \sin(\theta) + Y \cos(\theta)$$

- Biểu diễn hệ phương trình sang ma trận.

$$\begin{bmatrix} X' \\ Y' \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \begin{bmatrix} X \\ Y \end{bmatrix}$$

- Thực hiện rút gọn $\cos(\theta)$ ra làm nhân tử chung, ma trận xoay viết lại thành.

$$\begin{bmatrix} X' \\ Y' \end{bmatrix} = \cos(\theta) \begin{bmatrix} 1 & -\tan(\theta) \\ \tan(\theta) & 1 \end{bmatrix} \begin{bmatrix} X \\ Y \end{bmatrix}$$

- Việc tính toán trực tiếp ma trận này trên hệ thống máy tính kỹ thuật số đòi hỏi các bộ nhân phức tạp và tài nguyên để tính các hàm lượng giác $\cos(\theta)$, $\tan(\theta)$. Điều này giảm tốc độ xử lý và tăng diện tích silicon của vi mạch.

- Thay vì thực hiện quay vector một góc lớn θ trong một phép tính duy nhất, thuật toán chia quá trình này thành N bước lặp (tương ứng với cấu trúc N tầng pipeline). Tại mỗi tầng thứ i, hệ thống chỉ quay một góc định trước là α_i . Chiều quay tại mỗi bước được quyết định bởi hệ số $d_i \in \{-1, 1\}$.

$$\tan(\theta) = \tan(d_i \cdot \alpha_i) = d_i \cdot \tan(\alpha_i)$$

- Thay vào lại phương trình ta được:

$$X_{i+1} = X_i - d_i \cdot Y_i \cdot \tan(\alpha_i)$$

$$Y_{i+1} = Y_i + d_i \cdot X_i \cdot \tan(\alpha_i)$$

$$\tan(\theta_i) = 2^{-i}$$

- Bây giờ thuật toán bắt buộc phải chọn theo phương trình:

$$\tan(\alpha_i) = 2^{-i}$$

- Hay nói cách khác, các góc α_i được cố định cứng là:

$$\alpha_i = \arctan(2^{-i})$$

- Thay ngược vào phương trình ta được.

$$X_{i+1} = X_i - d_i \cdot Y_i \cdot 2^{-i}$$

$$Y_{i+1} = Y_i + d_i \cdot X_i \cdot 2^{-i}$$

- Hệ phương trình lặp.

$$X_{i+1} = X_i - d_i \cdot Y_i \cdot 2^{-i}$$

$$Y_{i+1} = Y_i + d_i \cdot X_i \cdot 2^{-i}$$

$$Z_{i+1} = Z_i - d_i \cdot \arctan(2^{-i})$$

- Hệ số suy hao: Ở phương trình ma trận rút nhân tử chung, tại mỗi bước xoay thứ i , độ lớn của vector đã bị nhân thêm một lượng là $\frac{1}{\cos(\alpha_i)}$. Dựa vào công thức lượng giác $\cos^2(\alpha) = \frac{1}{1 + \tan^2(\alpha)}$, hệ số phình to tại bước i là:

$$K_i = \sqrt{1 + \tan^2(\alpha_i)} = \sqrt{1 + 2^{-2i}}$$

- Sau N bước lặp (với thiết kế 20 tầng, $N=20$), tổng độ phình của vector là một hằng số hội tụ tĩnh:

$$K = \prod_{i=0}^{N-1} \sqrt{1 + 2^{-2i}} \approx 1.64676$$

- Sự gia tăng biên độ này là một hiệu ứng phụ tất yếu của việc đơn giản hóa phép toán tan. Do đó, để thu được giá trị sin và cos (hoặc biên độ thực của vector) chính xác tuyệt đối, ngõ ra cuối cùng của thuật toán (khối Post-processor) phải trải qua bước chuẩn hóa biên độ bằng cách nhân với hệ số bù A (Gain):

$$A = \frac{1}{K} \approx 0.60725$$

2.2.3. Chế độ xoay góc (Rotation mode)

- Trong chế độ này, bộ vi xử lý bắt đầu với một vector đầu vào $[x_0, y_0]$ và một góc xoay mục tiêu được nạp vào thanh ghi z_0 . Thuật toán sẽ thực hiện một chuỗi các phép xoay sao cho vector liên tục tiến về góc đích, đồng nghĩa với việc đẩy giá trị trong thanh ghi góc z hội tụ dần về mức 0. Biến giúp chúng ta định hướng d được đánh giá trực tiếp bằng việc lấy dấu của thanh ghi z_i tại mỗi bước.
- Điều kiện xác định chiều quay (d_i).

$$d_i = \begin{cases} 1 & \text{nếu } Z_i \geq 0 \\ -1 & \text{nếu } Z_i < 0 \end{cases}$$

- + $d = 1$ sẽ xoay ngược chiều kim đồng hồ để giảm pha
- + $d = -1$ sẽ xoay cùng chiều kim đồng hồ để tăng pha
- Hệ phương trình lặp.

$$X_{i+1} = X_i - d_i \cdot Y_i \cdot 2^{-i}$$

$$Y_{i+1} = Y_i + d_i \cdot X_i \cdot 2^{-i}$$

$$Z_{i+1} = Z_i - d_i \cdot \arctan(2^{-i})$$

- Chế độ Rotation trong hệ tọa độ tròn là cơ sở để thiết kế các bộ tạo dao động kỹ thuật số. Bằng cách khởi tạo $x_0 = 1/K$ (nghịch đảo của hệ số tỷ lệ), $y_0 = 0$ và $z_0 = \alpha_0$. Sau N vòng lặp, đầu ra cuối cùng sẽ trả về trực tiếp $x_N = \cos(\alpha)$ và $y_N = \sin(\alpha)$.

2.2.4. Chế độ tìm góc (Vectoring mode)

- Thay vì xoay theo một góc cho trước, chế độ Vectoring khởi tạo với một vector không xác định $[x_0, y_0]$ và thanh ghi góc $z_0 = 0$. Mục tiêu của thuật toán là quay vector này áp sát vào trục hoành dương (trục x), đồng nghĩa với việc làm cho thành phần tọa độ y hội tụ về 0. Tại chế độ này, hướng xoay d_i được đánh giá dựa vào dấu của thành phần y_i
- Điều kiện xác định chiều quay (d_i).

$$d_i = \begin{cases} -1 & \text{nếu } Y_i \geq 0 \\ 1 & \text{nếu } Y_i < 0 \end{cases}$$

- + $d = 1$ -> Thực hiện xoay ngược chiều kim đồng hồ để kéo y lên mức 0.
- + $d = -1$ -> Thực hiện xoay cùng chiều kim đồng hồ để đẩy y xuống mức 0.
- Hệ phương trình lặp.

$$X_{i+1} = X_i - d_i \cdot Y_i \cdot 2^{-i}$$

$$Y_{i+1} = Y_i + d_i \cdot X_i \cdot 2^{-i}$$

$$Z_{i+1} = Z_i - d_i \cdot \arctan(2^{-i})$$

- Sau N bước lặp, đầu ra x_N sẽ chứa đựng biên độ của vector nguyên bản được nhân lên với hệ số tỷ lệ K, trong khi thành ghi góc z_N sẽ lưu trữ chính xác góc pha gốc ban đầu $\alpha_0 = \arctan(y_0 / x_0)$.

2.3. Chuẩn biểu diễn số phẩy tĩnh (Fixed-Point)

2.3.1. Sự cần thiết cho kiến trúc FPGA

- Trong các hệ thống vi xử lý máy tính đa dụng, các số thực thường được tính toán dưới chuẩn số phẩy động (Floating Point) theo tiêu chuẩn IEEE 754. Việc hiện thực hóa các bộ cộng/nhân phẩy động trên FPGA đòi hỏi khối lượng tài nguyên khổng lồ và tốn hàng chục chu kỳ xung nhịp để hoàn thành một phép toán.
- Do mục tiêu của đồ án là thiết kế một bộ tăng tốc phần cứng tối ưu về tốc độ và diện tích, toàn bộ hệ thống CORDIC được thiết kế để hoạt động bằng chuẩn Số phẩy tĩnh (Fixed-Point).
- Trong chuẩn này, vị trí của dấu phẩy thập phân được cố định tại một ranh giới bit xác định. Nhờ đó, phần cứng có thể thực hiện các phép cộng/trừ số thực bằng chính các bộ cộng số nguyên truyền thống, giúp đạt tốc độ $F_{\max}$ cao với độ trễ (Latency) thấp.

2.3.2. Định dạng Q2.29 cho trục tọa độ

- Đối với dữ liệu độ lớn vector và các hàm cos/sin, dải giá trị ngõ vào là 1.0 và ngõ ra trong khoảng $[-1.0, 1.0]$. Tuy nhiên, trong quá trình chạy 20 vòng lặp bên trong lõi CORDIC, độ lớn của vector sẽ liên tục bị giãn ra ở mức xấp xỉ ± 1.64676 (hệ số Gain) trước khi được khối Hậu xử lý bù trừ lại.
- Do đó nhóm sẽ phân bổ chuẩn Q2.29 tức 1 bit dấu, 2 bit nguyên, 29 bit thập phân cho các tín hiệu x_i, y_i, x_o, y_o . Với 2 bit phần nguyên cùng 1 bit dấu, hệ thống có thể lưu trữ các giá trị trong khoảng $[-4.0, 3.99]$, nên nếu trong trường hợp xấu nhất phải nhân với hệ số giãn 1.64676 thì cũng vẫn nằm trong khoảng này.
- Công thức quy đổi giá trị thập phân sang số nguyên định dạng Q2.29:

$$\text{Số phẩy tĩnh} = \text{round} (\text{Số thực} * 2^{n_{\text{bit}}})$$

- Ví dụ: Để có giá trị 1.0 ta sẽ quy đổi ra số nguyên vào mạch là $1.0 * 2^{29} = 536870912$.

2.3.3. Ánh xạ cho trục góc Z

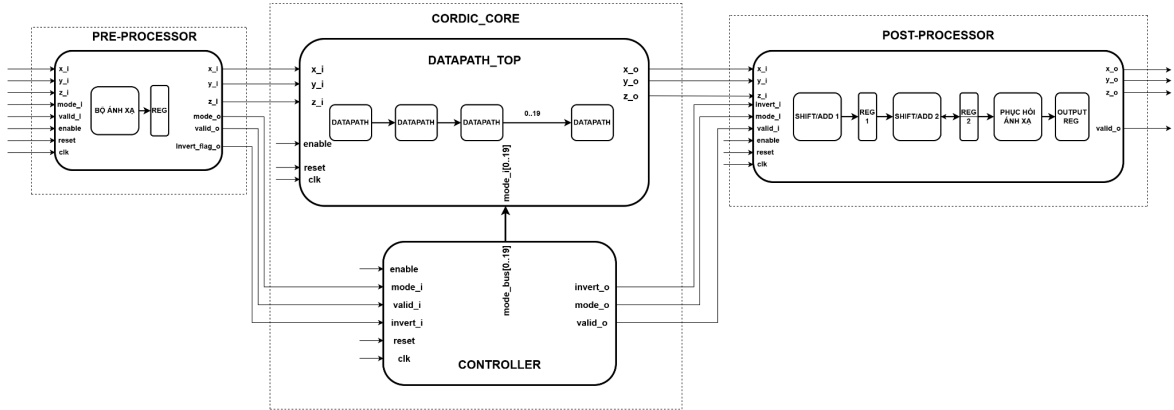
- Toàn bộ vòng tròn lượng giác 360° được ánh xạ vào 32 bit của thanh ghi dùng số bù 2 có giá trị trong khoảng $[-2^{31} ; +2^{31} - 1]$.
- Hoạt động giống như chuẩn số bù 2 của phần cứng. VD: Góc $+181^\circ$ vượt quá $+180^\circ$ thì thực chất nó là góc 179° tương tự như việc bị overflow từ $+2^{31} - 1$ về -2^{31}
- Công thức chuyển đổi góc bất kì (α) về hệ số nguyên 32 bit:

$$\text{Số nguyên} = \text{round} \left(\alpha * \frac{2^{31}}{180} \right)$$

- + Giá trị $+180^\circ$ tương ứng với $+2^{31} - 1 = 2147483647$.
- + Giá trị -180° tương ứng với $-2^{31} = -2147483648$.
- + Giá trị $+90^\circ$ tương ứng với $90 * (2^{31} / 180) = 2^{30} = 1073741824$.

Chương 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

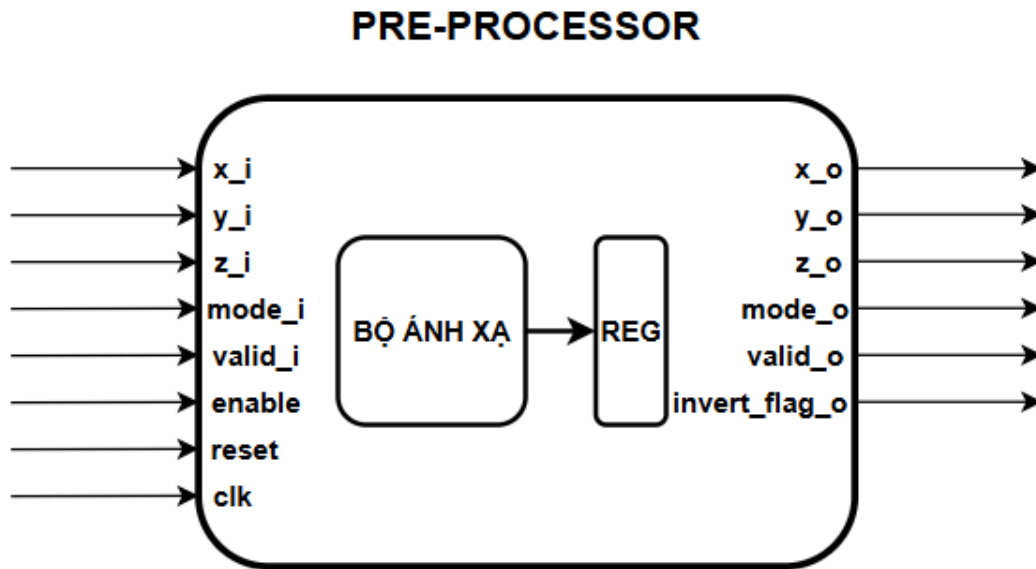
3.1. Sơ đồ khối tổng quát



- Khối thực hiện tính toán thuật toán CORDIC bao gồm các khối: PRE-PROCESSOR, DATAPATH_TOP, CONTROLLER, POST-PROCESSOR.
- Dữ liệu cần tính toán sẽ được đưa vào khối PRE-PROCESSOR và sau khi được tính toán xong thì sẽ cho ra kết quả ở khối POST-PROCESSOR kèm với tín hiệu $valid_o$ báo hiệu đã có kết quả.
- Tín hiệu $enable$ sẽ được nhận từ các khối bên ngoài, có thể là tín hiệu cho phép ghi vào bộ nhớ chứa kết quả hoặc cho phép truyền kết quả vào một khối nào khác trong một hệ thống lớn. Tín hiệu $enable$ cho phép thực hiện ghi các giá trị đã được tính toán ở từng tầng ghi vào các thanh ghi pipeline nếu tín hiệu $enable$ này không bật thì tương đương với việc dữ liệu sẽ không được truyền qua các thanh ghi và sẽ không xuất ra kết quả.
- Tín hiệu $valid_i$ là tín hiệu thông báo là dữ liệu đã sẵn sàng và được đẩy vào khối tiền xử lý và nó được tận dụng để làm tín hiệu $valid_o$ sau khi thực hiện xong tính toán. Tín hiệu $valid_o$ sẽ thực hiện dịch chuyển qua các thanh ghi ở khối PRE-PROCESSOR, CONTROLLER, POST-PROCESSOR nó sẽ dịch chuyển cùng lúc với dòng chảy dữ liệu đang được xử lý của nó.
- Tín hiệu clk là dùng cho xung nhịp của cả hệ thống.

- Tín hiệu reset dùng để đặt các thanh ghi ở trong hệ thống trở về trạng thái ban đầu là bằng 0.

3.2. Thiết kế khối tiền xử lý (Preprocessor)



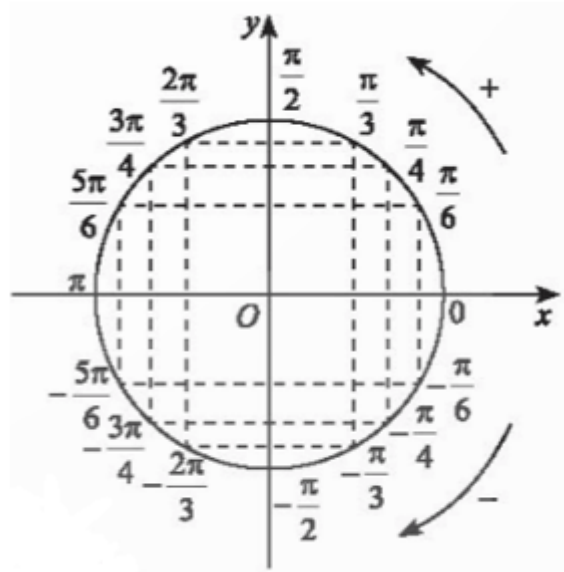
3.2.1. Chức năng

- Trong thuật toán CORDIC lỗi tồn tại một vấn đề đó là chuỗi các góc quay hằng số trong bảng LUT $\arctan(2^{-i})$ giảm dần và có tổng giới hạn hội tụ tối đa chỉ đạt xấp xỉ 99.88° . Điều này khiến cho lỗi CORDIC thông thường chỉ có thể tính toán chính xác khi các góc ngõ vào cả trong chế độ Rotation và Vectoring nằm ở nửa bên phải trong vòng tròn lượng giác, tức góc phần tư thứ 1 và thứ 4 $[-90^\circ; +90^\circ]$.
- Để giải quyết vấn đề trên, ta sẽ mở rộng dải hoạt động của toàn hệ thống ra trọn vẹn vòng tròn lượng giác 360° . Vì thế khối Tiền xử lý (Preprocessor) được nhóm thiết kế để thực hiện điều đó.
- Khối này có nhiệm vụ phát hiện các góc lớn hoặc các vector nằm ở góc phần tư thứ 2 và thứ 3 để thực hiện phép toán dời trục tọa độ qua kỹ thuật lấy đối xứng góc để đưa dữ liệu về vùng hội tụ an toàn của lỗi CORDIC (tức góc

phần tư thứ nhất và thứ tư), đồng thời phát tín hiệu cờ báo hiệu cho khối Hậu xử lý (Postprocessor) ở cuối hệ thống biết để hiệu chỉnh lại kết quả.

3.2.2. Nguyên lí hoạt động

- Input:
 - **x_i, y_i, z_i**: Giá trị tọa độ x,y và góc pha z của ngõ vào.
 - **mode_i**: Tín hiệu chọn chế độ giữa Rotation (Mode_i = 1) và Vectoring (Mode_i = 0).
 - **valid_i**: Cờ báo tín hiệu của gói dữ liệu.
 - **Clk, enable, reset**: xung clock và các tín hiệu cho phép hoạt động, khởi tạo hệ thống.
- Output:
 - **x_o, y_o, z_o**: Giá trị tọa độ ngõ ra x, y và góc pha z đã qua xử lí.
 - **mode_o**: được truyền từ ngõ vào mode_i để đồng bộ với luồng dữ liệu.
 - **valid_o**: được truyền từ ngõ vào valid_i đi trễ 1 chu kì xung nhịp, giúp lõi CORDIC Core biết khoảng khắc dữ liệu đã được ổn định để bắt đầu tính toán.
 - **invert_flag_d**: cờ báo hiệu các input có được xử lí hay không, tín hiệu này sau đó sẽ được khối Hậu xử lí sử dụng.



- Phép biến đổi ở khối này sẽ dựa trên kỹ thuật đối xứng góc toán học trong vòng tròn lượng giác.
- Đầu tiên khối lựa chọn chế độ đang tính toán dựa trên tín hiệu **mode_i**.
- Ở chế độ **Rotation (mode_i=1)**, hệ thống so sánh góc nạp vào **z_i** với tham số góc 90° . Nếu lớn hơn 90° , mạch tổ hợp sẽ trừ đi một góc 180° và kích hoạt cờ **invert_flag_d** = 1 báo hiệu đã xử lý góc để cho khối Hậu xử lý biết sau này. Còn bé hơn -90° , mạch lấy góc đó cộng thêm 180° đồng thời cũng bật cờ **invert_flag_d** lên luôn. Đối với các góc nằm trong vùng an toàn thì không cần phải làm gì hết và giữ nguyên cờ **invert_flag_d**.
- Ở chế độ **Vectoring (mode_i=1)**, mạch sẽ kiểm tra xem bit dấu lớn nhất của tọa độ **x_i** ngõ vào để xác định dấu của vector có phải nằm ở nửa mặt phẳng bên trái hay không. Nếu tọa độ **x_i** mang giá trị âm, có nghĩa là vector đang thuộc góc phần tư thứ 2 hoặc 3, khối sẽ đảo dấu tọa độ x,y để quay đối xứng vector qua gốc tọa độ sang nửa mặt phẳng bên phải, đồng thời kích hoạt cờ **invert_flag_d**. Còn nếu tọa độ **x_i** dương rồi thì sẽ được giữ nguyên và đi thẳng qua khối này.

- Sau khi ta đã có được kết quả, các ngõ ra output sẽ đi vào tầng thanh ghi Flipflop và chỉ được đi qua ở cạnh lên xung clock và tín hiệu enable được kích hoạt.

3.3. Thiết kế lõi CORDIC

3.3.1. Bảng tra hằng số góc (LUT)

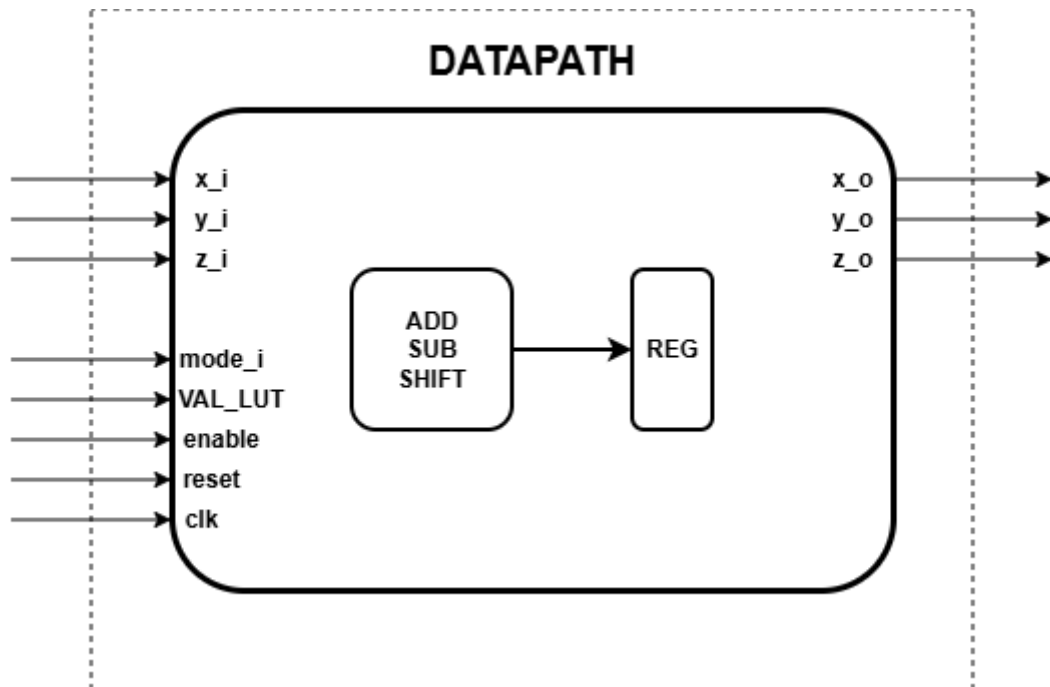
- Bảng tra hằng số góc là nơi lưu trữ giá trị của góc xoay tại mỗi tầng của thuật toán CORDIC là $\alpha_i = \arctan(2^{-i})$.
- Sử dụng hệ thống góc chuẩn hóa toàn dải với quy ước nửa vòng tròn (180° hoặc $\pi \text{ rad}$) tương đương với giá trị lớn nhất của số có dấu 32-bit là 2^{31} .
- Công thức chuyển đổi: Giá trị $LUT_i = \arctan(2^{-i}) \times \frac{2^{31}}{180}$
- Ví dụ: Tầng đầu tiên, $i = 0$,

$$\arctan(2^{-0}) = 45; 45 \times \frac{2^{31}}{180} = 536870912$$

Tầng (Stage i)	Độ	Giá trị 32-bit thập phân
0	≈ 45	536870912
1	≈ 26.56505	316933406
2	≈ 14.03624	167458907
3	≈ 7.12502	85004756
4	≈ 3.57633	42667331
5	≈ 1.78991	21354465
6	≈ 0.89517	10682178
7	≈ 0.44761	5341795
8	≈ 0.22381	2671073

9	≈ 0.11191	1335558
10	≈ 0.05595	667782
11	≈ 0.02798	333892
12	≈ 0.01399	166946
13	≈ 0.00699	83473
14	≈ 0.00350	41737
15	≈ 0.00175	20868
16	≈ 0.00087	10434
17	≈ 0.00044	5217
18	≈ 0.00022	2609
19	≈ 0.00011	1304

3.3.2. Khối DATAPATH.



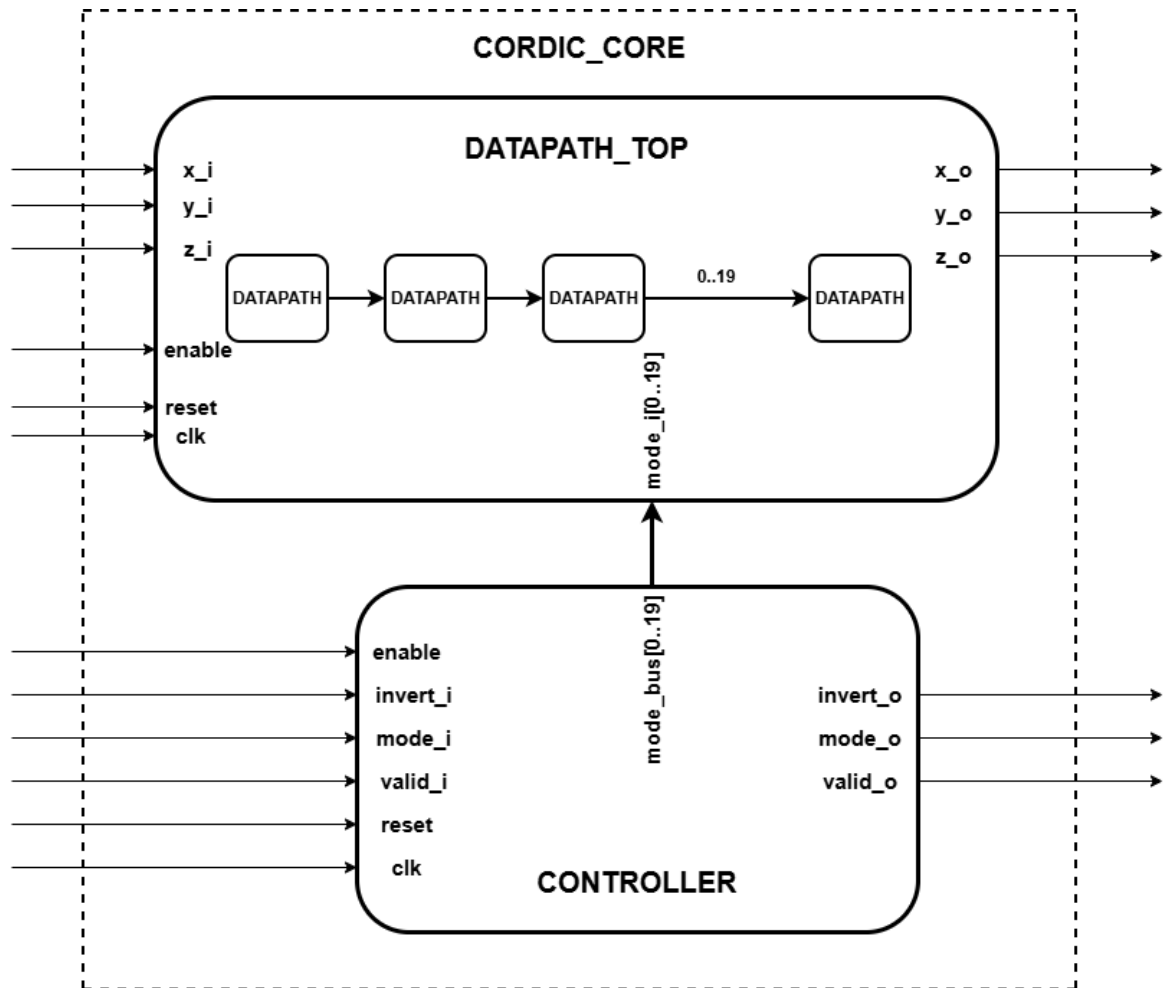
3.3.2.1. Chức năng của khối DATAPATH.

- Trong thuật toán CORDIC sẽ có hai chế độ để chúng ta thực hiện tính toán: Rotation và Vectoring.
- Với mỗi chế độ, mỗi tầng (mỗi lần xoay) thì sẽ có cách cộng/trừ và dịch bit khác nhau.
- Thiết kế DATAPATH này được thiết kế để dùng chung cho các trường hợp đây, với mỗi trường hợp chúng ta sẽ thực hiện truyền các thông số khác nhau cho nó thực hiện tính toán.

3.3.2.2. Nguyên lý hoạt động.

- INPUT:
 - x_i, y_i, z_i : Nhận giá trị đầu vào x, y, z từ khối tiền xử lý.
 - $mode_i$: Tín hiệu mode nhận từ controller.
 - VAL_LUT : Nhận giá trị từ bảng LUT được tính toán sẵn theo từng tầng.
 - $enable$: Tín hiệu cho phép ghi vào thanh ghi.
 - $reset, clk$.
- OUTPUT:
 - x_o, y_o, z_o : Giá trị của x, y, z sau khi được tính toán và lưu vào thanh ghi thì sẽ được đẩy ra từ ba đường tín hiệu này.
- Khối **DATAPATH** sẽ thực hiện tính toán sau khi nhận đầu vào x, y, z và xác định chế độ thông qua $mode_i$ và nhận giá trị **VAL_LUT** tương ứng với từng tầng. Rồi thực hiện ghi vào thanh ghi để chuyển sang tầng tiếp theo.
- Khối **DATAPATH** này được coi như là một tầng trong **DATAPATH_TOP**, khối này sẽ được generate thành 20 khối tương ứng với 20 lần xoay của thuật toán **CORDIC**.

3.3.3. Khối CORDIC_CORE



3.3.3.1. Chức năng của khối CORDIC_CORE.

(a) Chức năng của khối DATAPATH_TOP.

- Khối DATAPATH_TOP đóng vai trò là lõi xử lý của hệ thống. Chức năng chính của khối là tạo một hệ thống đường ống (pipeline) 20 tầng bằng cách nhân bản và ghép nối 20 module DATAPATH đơn lẻ. Khối thực hiện tiếp nhận đồng thời luồng dữ liệu tọa độ từ khối Tiền xử lý và tín hiệu chế độ (mode_bus) từ khối Điều khiển, sau đó thực thi 20 vòng lặp xoay vector liên tiếp trước khi xuất dữ liệu đã hội tụ sang khối Hậu xử lý để hoàn thiện.

(b) Chức năng của khối CONTROLLER.

- Khối CONTROLLER đóng vai trò là bộ quản lý đồng bộ thời gian của hệ thống CORDIC. Chức năng chính của nó là tiếp nhận các tín hiệu cờ trạng thái ($mode_i$, $invert_i$, $valid_i$) từ khối PRE-PROCESSOR để thực hiện hai nhiệm vụ song song: (1) Phân phối tín hiệu điều khiển chế độ ($mode_bus$) cho 20 tầng tính toán của khối $datapath_top$, và (2) tạo trễ (delay) các tín hiệu cờ này để xuất cho khối POST-PROCESSOR, đảm bảo luồng tín hiệu điều khiển luôn đi kèm chính xác với gói dữ liệu tọa độ (X, Y, Z) tương ứng.

3.3.3.2. Nguyên lý hoạt động của CORDIC_CORE

(a) Nguyên lý hoạt động của DATAPATH_TOP

- Nguyên lý hoạt động của DATAPATH_TOP dựa trên cơ chế đường ống (pipeline). Dữ liệu không lan truyền xuyên suốt toàn bộ hệ thống trong một chu kỳ duy nhất, mà được luân chuyển tuần tự qua 20 tầng xử lý theo từng nhịp xung (clock). Tại mỗi tầng, mạch tính toán đã được mã hóa cứng các hằng số riêng biệt bao gồm biên độ dịch bit (SHIFT) và giá trị góc lượng giác (VAL_LUT). Khi luồng dữ liệu (X, Y, Z) dịch chuyển đến một tầng bất kỳ, mạch tổ hợp tại đó sẽ kết hợp với các hằng số này để thực hiện một bước xoay vector, sau đó chốt kết quả vào thanh ghi để truyền sang tầng kế tiếp ở chu kỳ clock tiếp theo.

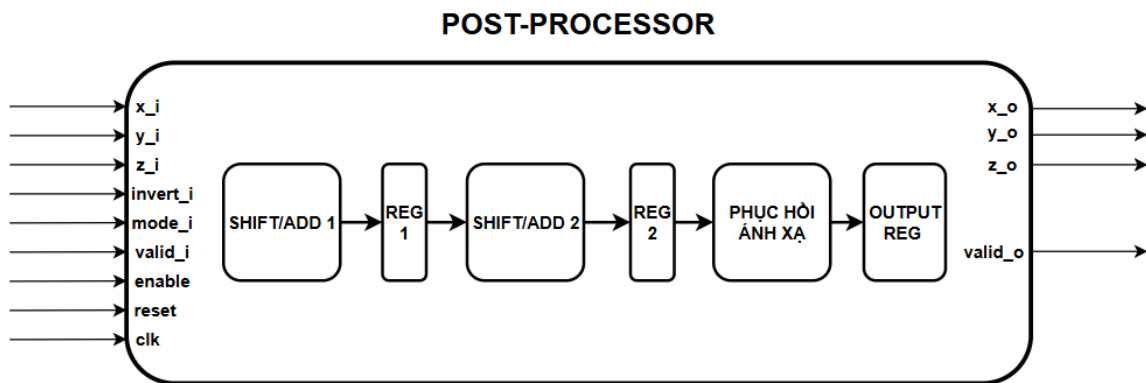
(b) Nguyên lý hoạt động của CONTROLLER

- Nguyên lý hoạt động của khối CONTROLLER dựa trên cấu trúc các thanh ghi dịch độ dài 20 bit, tương ứng với độ trễ 20 chu kỳ xung nhịp của khối $datapath_top$.
- Mỗi khi có một gói dữ liệu mới đi vào lõi CORDIC, các cờ trạng thái ($invert$, $mode$, $valid$) đi kèm với gói đó sẽ được đẩy vào thanh ghi dịch. Vì khối POST-PROCESSOR ở ngõ ra bắt buộc phải biết gói dữ liệu (X, Y, Z) vừa tính xong thuộc chế độ nào (Rotation hay Vectoring) và có bị đổi dấu hay không để thực hiện khôi phục góc phân tư, các thanh ghi này sẽ "giữ" và

“dịch” chuyển các cờ trạng thái đi chậm lại đúng 20 nhịp clock. Kết quả là tại ngõ ra, các cờ này sẽ xuất hiện đồng thời và khớp nối hoàn hảo về mặt thời gian (timing) với dữ liệu đã tính toán xong.

- Riêng đối với tín hiệu mode, khối điều khiển áp dụng cơ chế đi tắt bằng cách ghép nối tín hiệu ngõ vào trực tiếp với 19 bit lấy từ thanh ghi dịch để tạo thành bus điều khiển 20-bit, đảm bảo mỗi tầng của DATAPATH_TOP đều nhận được đúng bit cấu hình của gói dữ liệu mà nó đang xử lý.

3.4. Thiết kế khối hậu xử lý (Postprocessor)



3.4.1. Chức năng

- Sau khi dữ liệu đi qua 20 tầng pipeline của lõi CORDIC, kết quả thu được vẫn chưa phải là kết quả toán học cuối cùng. Do bản chất của thuật toán CORDIC tại mỗi vòng lặp, độ dài của vector bị giãn ra một tỷ lệ $\sqrt{1 + 2^{-2i}}$. Sau 20 vòng lặp, tổng độ giãn đại diện bằng hằng số Gain (A) sẽ có giá trị xấp xỉ 1.64676. Do đó, để chúng ta có thể thu được giá trị thực, ngõ ra bắt buộc phải được nhân với hệ số bù K được tính bằng công thức

$$K \approx \frac{1}{A_{20stages}} \approx \frac{1}{1.64676} \approx 0.60725$$

- Khối Hậu xử lý được nhóm thiết kế nhằm thực hiện hai nhiệm vụ sau:

- + Thực hiện bù lại hệ số K được biểu diễn dưới dạng số nguyên 32 bit để đưa biên độ vector về đúng với tỷ lệ thực tế.
- + Dựa vào cờ trạng thái **invert_flag_d** được truyền từ khối Tiền xử lí, ta sẽ đảo dấu lại trục tọa độ hoặc bù lại cộng/trừ 180° nếu cờ được bật lên 1. Để trả kết quả lại về đúng góc phần tư ban đầu trên vòng tròn lượng giác mà ta đã thực hiện tính toán thay đổi ở khối Tiền xử lí.

3.4.2. Nguyên lí hoạt động

- **Input:**

- **x_i, y_i, z_i:** Kết quả từ tầng thứ 20 của lõi CORDIC, chưa qua xử lí lại.
- **invert_i:** Tín hiệu từ cờ **invert_flag_d** của khối Tiền xử lí truyền qua 20 tầng CORDIC, khối Hậu xử lí dựa vào đây để quyết định có đảo chiều/ bù góc hay không.
- **Mode_i:** Tín hiệu cho biết đang ở mode Rotation hay Vectoring.
- **valid_i:** Được truyền từ lõi báo hiệu dữ liệu tính toán đã sẵn sàng
- **Clk, reset, enable:** Các tín hiệu đồng bộ hóa 2 tầng thanh ghi bên trong khối. enable cho phép đóng/mở luồng dữ liệu, reset hoạt động ở mức cao sẽ đưa các ngõ ra về 0.

- **Output:**

- **x_o, y_o, z_o:** Các ngõ ra chính thức của toàn bộ hệ thống sau khi đã qua bước xử lí.
- **valid_o:** Tín hiệu sau khi qua chu kì trễ nội bộ của khối Hậu xử lí, báo hiệu kết quả cuối cùng đã hợp lệ.

- Tương tự như khối Tiền xử lí, khối sẽ dựa vào tín hiệu **mode_i** để biết đang ở chế độ nào.
- Trong mode **Vectoring (mode_i=1)**: Ép tung độ y ngõ ra bằng 0, do tính chất vector được áp sát vào trục hoành. Đối với giá trị x, ta nhân bù tỉ lệ với hệ số

Gain (A) để có được tọa độ x thực tế. Nếu khối nhận được cờ invert_i = 1 từ khối Tiền xử lí thì sẽ tiến hành bù cộng/ trừ 180° cho góc z hiện tại để trả về đúng với góc thực tế của vector. Nếu không có cờ thì giữ nguyên kết quả.

- Tuy nhiên, ở đây ta sẽ không sử dụng bộ nhân multiplier do yêu cầu của đề tài nên ta sẽ chỉ sử dụng phép dịch bit và bộ cộng/ trừ để thay thế.
- Hệ số K chúng ta đang là $K \approx 0.607252935$, ta sẽ chuyển đổi sang một chuỗi phép tính cộng trừ các lũy thừa của 2 với sai số nhỏ như sau:

$$K \approx 2^{-1} + 2^{-3} - 2^{-6} - 2^{-9} - 2^{-13} - 2^{-15} - 2^{-16} \approx 0.607254$$

- Việc nối tiếp nhiều bộ cộng/trừ 32-bit trong cùng một chu kỳ xung nhịp tạo ra hiện tượng nút thắt cổ chai khiến cho dòng điện mang dữ liệu mất nhiều thời gian lan truyền qua hệ thống cổng logic, khiến tần số tối đa $F_{\max}$ của mạch bị sụt giảm nghiêm trọng.
- Để khắc phục, nhóm thiết kế đã thực hiện chia chuỗi ra làm hai và phân bổ vào 2 nhịp xung clock nối tiếp nhau:
- + Tầng 1: Thực hiện tính toán chuỗi thứ nhất ($2^{-1} + 2^{-3} - 2^{-6} - 2^{-9}$) và chuỗi thứ hai ($2^{-13} + 2^{-15} + 2^{-16}$). Sau đó kết quả được gán vào hai thanh ghi trung gian giảm thời gian lan truyền.
- + Tầng 2: Thực hiện phép trừ 2 chuỗi trên lại để gộp lại với nhau về chuỗi cũ, hoàn thiện việc nhân bù hệ số K.
- Trong mode **Rotation (mode_i=0)**: Nếu có cờ invert_i tức góc nạp vào đang ở ngoài khoảng $\pm 90^\circ$, ta sẽ đảo dấu của cả x và y để khôi phục về lại đúng giá trị hàm cos, sin trong các góc phần tư thứ 2, 3.
- Sau khi khối Hậu xử lí kết thúc, các dữ liệu output **x_o, y_o, z_o** và cờ **valid_o** sẽ được chốt ngay tại xung cạnh lên và đi ra ngoài khối.

Chương 4. MÔ PHỎNG VÀ ĐÁNH GIÁ HIỆU NĂNG

4.1. Xây dựng testbench kiểm chứng

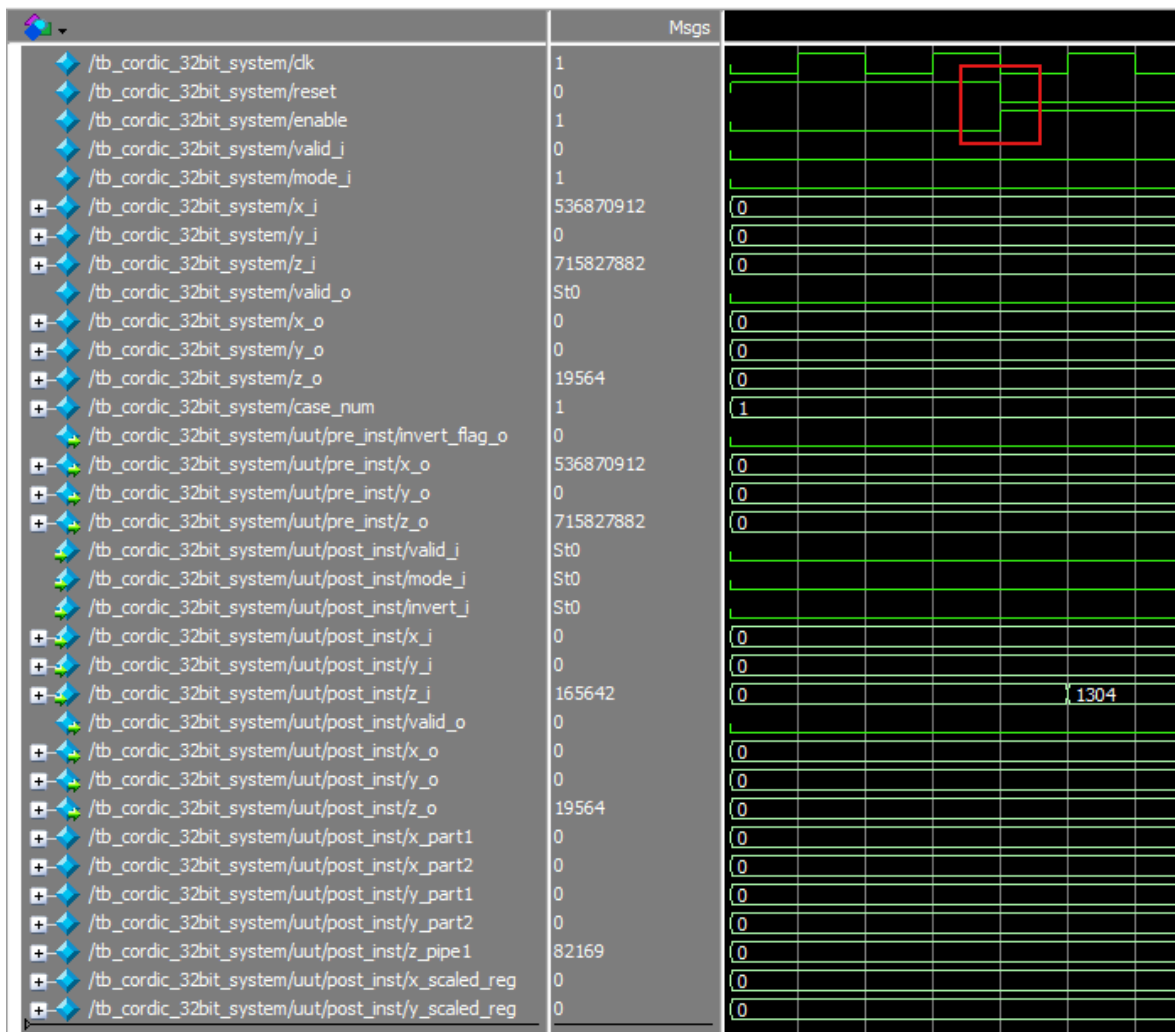
- **Giai đoạn 1:** Khởi động và Thiết lập.

- Hệ thống được cấp xung nhịp clk có chu kỳ là $T = 10\text{ns}$ ($f = 100\text{MHz}$), đảo trạng thái mỗi 5ns .
- Tại thời điểm $t = 0\text{ns}$, toàn bộ tín hiệu đầu vào sẽ được ép về 0, riêng tín hiệu $\text{reset} = 1$ để xóa sạch các thanh ghi bên trong.
- Sau đó 20ns , reset được kéo xuống 0 và enable được bật lên 1 để cho mạch bắt đầu hoạt động.
- Sau đó thực hiện đợi thêm 3 chu kỳ xung nhịp để hệ thống ổn định trước khi bắt đầu thực hiện tính toán.
- **Giai đoạn 2:** Truyền dữ liệu vào (4 test case).
 - Sử dụng task `send_input` để đóng gói việc gửi dữ liệu. Mỗi lần gọi task, dữ liệu sẽ được đẩy vào mạch trong đúng 1 chu kỳ clk (kèm theo cờ $\text{valid}_i = 1$), sau đó hạ cờ xuống. Các test case được bơm vào lần lượt cách nhau 5 chu kỳ clk.
 - Test Case 1 (Rotation - Tính lượng giác góc 60°):
 - $\text{mode}_i = 1$ (Rotation).
 - Mạch được yêu cầu xoay vector có độ dài $X = 1.0$ (bản chất số 536870912 chính là 1.0 trong định dạng dấu phẩy tĩnh Q2.29), góc $Z = 60^\circ$.
 - Mục tiêu: Tính ra $\cos(60^\circ)$ ở ngõ X_{out} và $\sin(60^\circ)$ ở ngõ Y_{out} .
 - Test Case 2 (Vectoring - Dò góc phần tư thứ I)
 - Đợi 5 chu kỳ sau Test Case 1, truyền tiếp Test Case 2.
 - Chế độ: $\text{mode}_i = 0$ (Vectoring).
 - Đưa vào một vector có tọa độ $(1.0, 1.0)$.
 - Mục tiêu: Mạch phải ép Y về 0 để tính ra độ lớn vector ≈ 1.414 ở X_{out} và góc $\approx 45^\circ$ ở ngõ Z_{out} .
 - Test Case 3 (Rotation - Góc từ 150° ở góc phần tư II)
 - Chế độ: $\text{mode}_i = 1$.
 - Truyền $X = 1, Y = 0, Z = 150$.

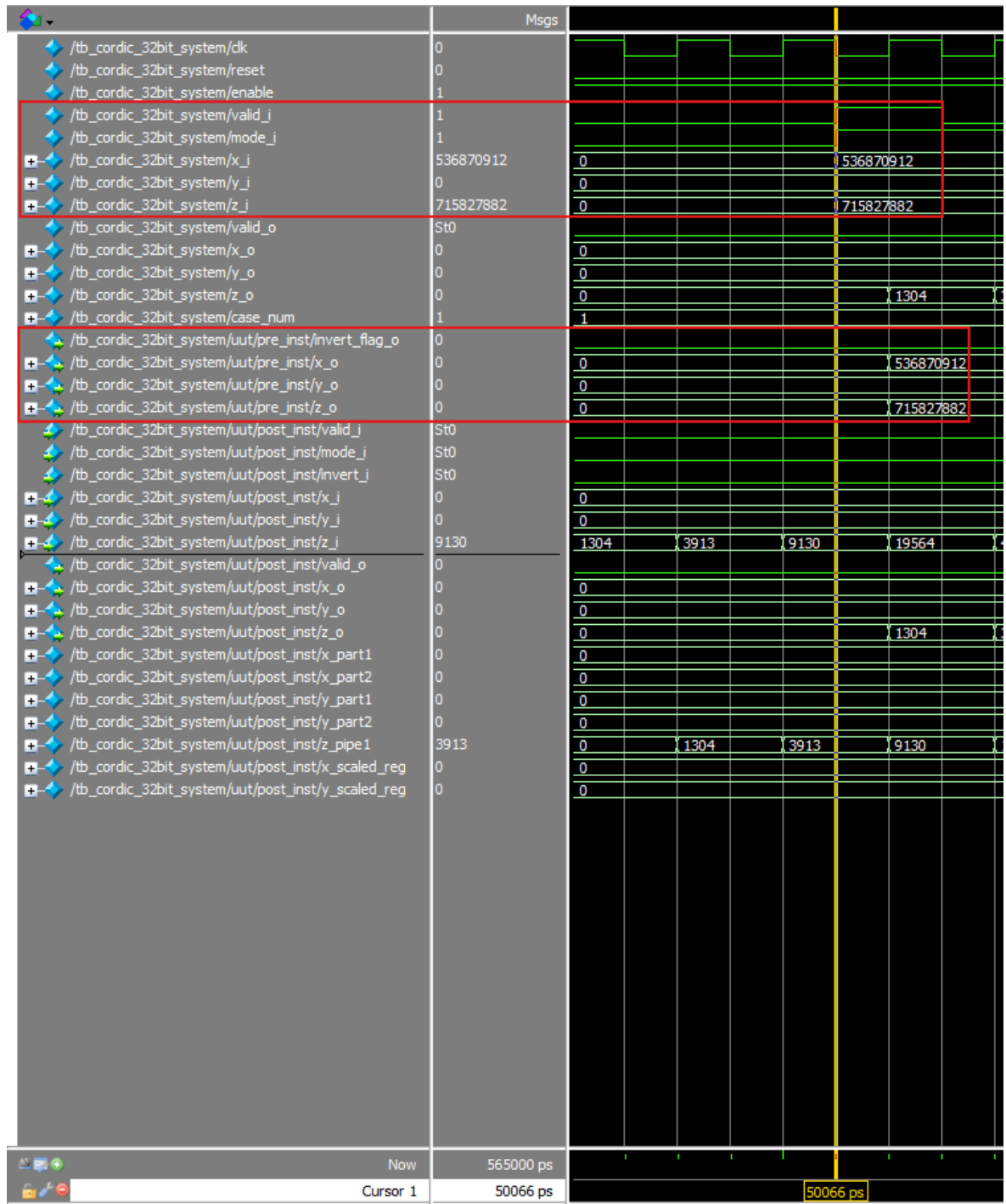
- Mạch pre_processor sẽ phải chuyển góc này về dải ± 90 trước khi đưa vào lõi CORDIC.
- Mục tiêu tính ra là $X_{out} \approx -0.86$, $Y \approx 0.5$
- Test Case 4 (Vectoring - Tọa độ âm ở góc phần tư thứ II)
 - Chế độ: $mode_i = 0$.
 - Đưa vào vector có tọa độ $(-1.0, 1.0)$. Hệ thống sẽ phải phát hiện X âm để thực hiện ánh xạ khôi phục góc cho đúng.
 - Mục tiêu: $X_{out} \approx 1.414$ và $Z \approx 135^\circ$.

4.2. Đánh giá kết quả mô phỏng chức năng (Pre-simulation)

- Khởi động và thiết lập.

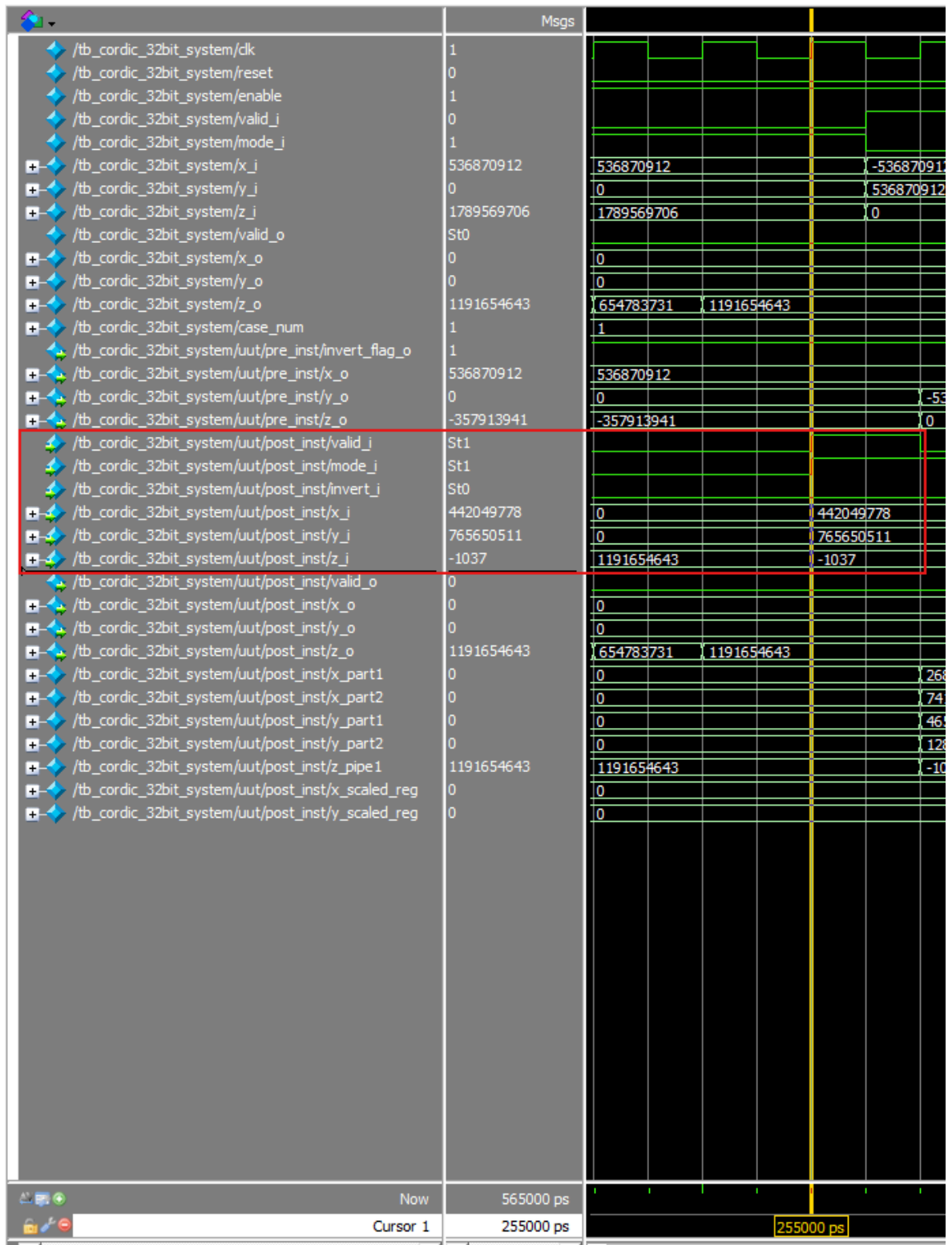


- Ở $T = 0\text{ns}$ tín hiệu reset đang được kéo lên mức 1 (mạch đang trong trạng thái reset) và tín hiệu enable (tín hiệu cho phép ghi các giá trị vào thanh ghi có trong mạch) đang ở mức 0. Đây là khoảng thời gian đang ở trạng thái reset.
- Kéo dài trạng thái đó từ $T = 0\text{ns}$ đến $T = 20\text{ns}$. Lúc này tín hiệu reset được kéo xuống 0 và tín hiệu enable được set lên 1, mạch đã bước vào trạng thái hoạt động và sẵn sàng nhận dữ liệu để thực hiện tính toán.
- **Test Case 1**
 - INPUT:
 - $x = 1 \Rightarrow x \approx 536870912$ (Đổi $x = 1$ số thực sang định dạng Q2.29)
 - $y = 0 \Rightarrow y \approx 0$ (Đổi $y = 0$ số thực sang định dạng Q2.29)
 - $z = 60 \Rightarrow z \approx 715827882$ (Đổi $z = 60$ số thực sang định dạng góc chuẩn hóa).
 - $\text{mode}_i = 1$: Rotation
 - $\text{valid}_i = 1$.



- Dữ liệu x, y, z, mode_i và valid_i đã sẵn sàng tại T = 50ns, lúc này dữ liệu đã đi qua khối tiền xử lý, vì test case 1 z = 60, nó không nằm trong góc phần tư thứ hai hay thứ ba thì dữ liệu đi khối tiền xử lý sẽ không thay đổi gì cả vì vậy cờ invert_flag_o cũng không được set lên 1.

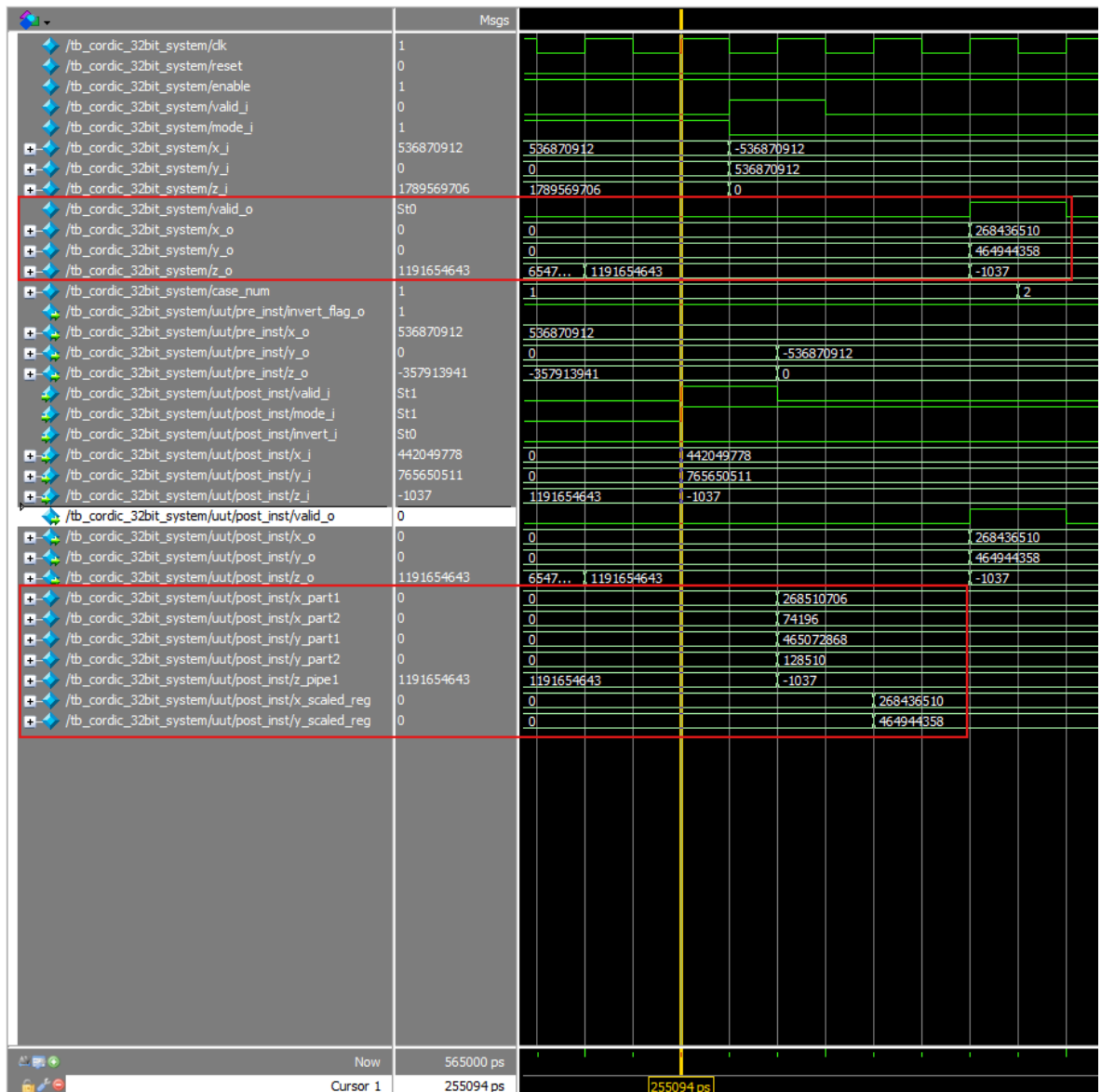
- Sau khi dữ liệu đi qua khối tiền xử lý và đợi đến clk cạnh lên tiếp theo tại $T = 55\text{ns}$ nó sẽ được ghi vào thanh ghi đứng sau của khối tiền xử lý và đầu ra của nó được nối thẳng với x_o , y_o , z_o các tín hiệu này sẽ được nối vào tầng 0 của DATAPATH_TOP để tiến hành tính toán.



- Sau khi dữ liệu đi vào khối DATAPATH_TOP phải cần đến 20 chu kỳ xung clk sau thì ta mới nhận được output của DATAPATH_TOP cùng

với lúc đó tín hiệu `valid_i`, `mode_i`, `invert_i` của phép tính này cũng được xuất ra từ khối CONTROLLER.

- Vì dữ liệu `x y z` chỉ mới được xử lý qua CORDIC_CORE nên có thể thấy `x_o`, `y_o`, `z_o` của khối DATAPATH_TOP lúc này là `x_i`, `y_i`, `z_i` của khối hậu xử lý:
 - $x_i = 442049778 \Rightarrow x \approx 0.823$ ($x = \cos(z) = \cos(60) = 0.5$).
Nhưng mà chúng ta có thể thấy độ dài của `x` đang bị dẫn ra gấp 1.64676 lần ($0.823 \approx 0.5 \times 1.64676$). Vì vậy chúng ta phải cần dữ liệu đi qua khối hậu xử lý để giải quyết việc đó.
 - $y_i = 765650511 \Rightarrow y \approx 1.426$ ($y = \sin(z) = \sin(60) = 0.866$).
Nhưng mà chúng ta có thể thấy độ dài của `y` đang bị dẫn ra gấp 1.64676 lần ($1.426 \approx 0.866 \times 1.64676$). Vì vậy chúng ta phải cần dữ liệu đi qua khối hậu xử lý để giải quyết việc đó.
 - $z_i = -1037 \Rightarrow z \approx -0.0000869$ (gần về 0) nó là giá trị của góc nên không bị dẫn. Việc `z` trừ dần về 0 thì đây là cách hoạt động của CORDIC.

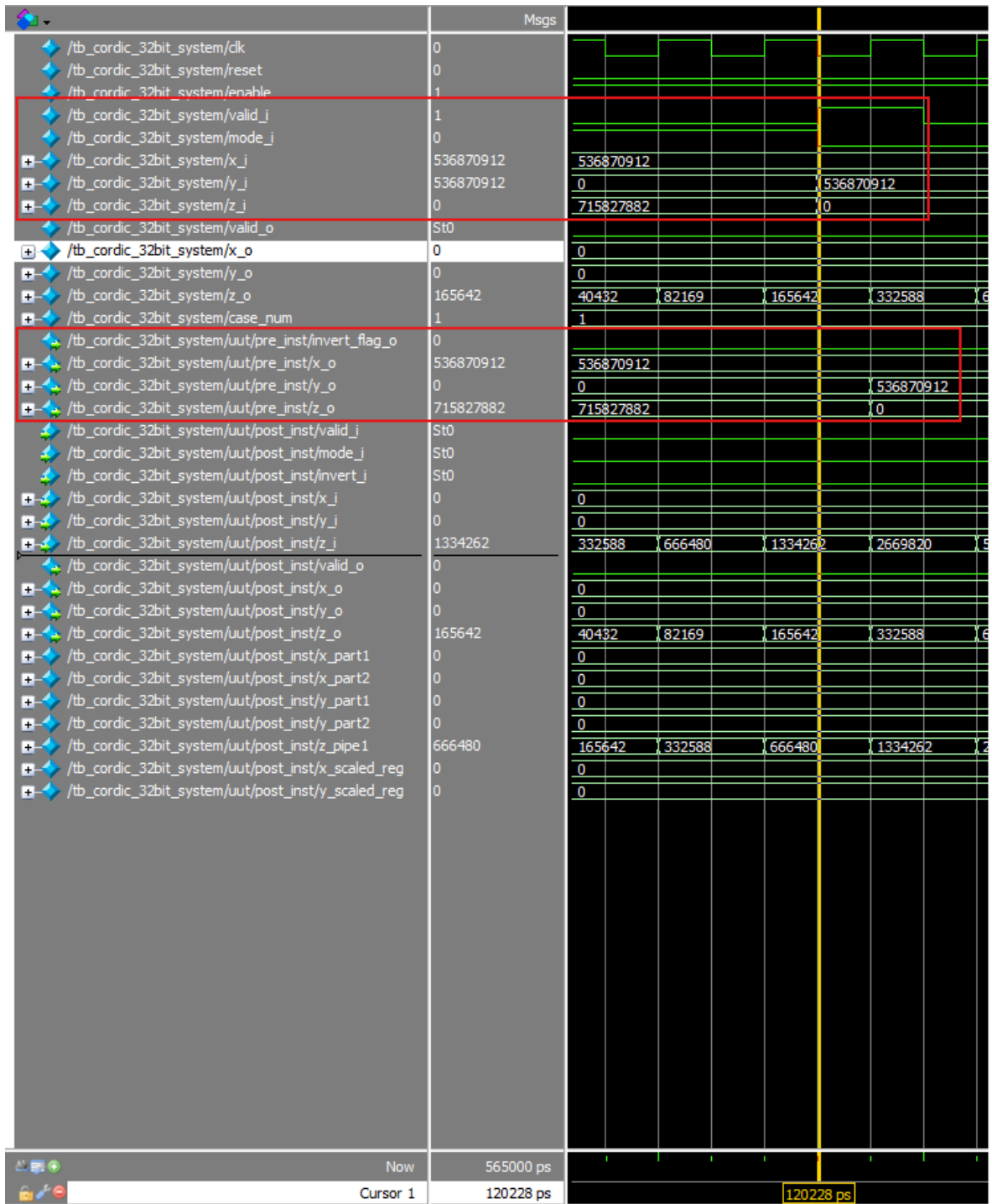


- Vì khối hậu xử lý thực hiện phép nhân được chia thành hai giai đoạn. Giai đoạn đầu tiên sẽ được xử lý và lưu vào x_part_1, x_part_2, y_part_1, y_part_2. Giai đoạn thứ hai sẽ hoàn thành xong việc đưa độ dẫn của x y về chiều dài ban đầu. Có thể thấy:
 - $x_scaled_reg = 268436510 \Rightarrow x \approx 0.5$ (đúng với kết quả mong đợi $\cos(60) = 0.5$).
 - $y_scaled_reg = 464944358 \Rightarrow y \approx 0.866$ (đúng với kết quả mong đợi $\sin(60) = 0.866$).

- Sau khi chuyển chiều dài x y về chiều dài ban đầu. Thì tầng tiếp theo là thực hiện ánh xạ lại. Nhưng mà do test case này có $invert = 0$ nên kết quả sẽ không thay đổi gì so với tầng thứ hai. Kết quả đó sẽ được ghi vào thanh ghi cuối cùng và xuất ra ngoài đồng thời tín hiệu $valid_o$ được set = 1. Ngõ ra của khối hậu xử lý cũng là ngõ ra của cả mạch.

- **Test case 2**

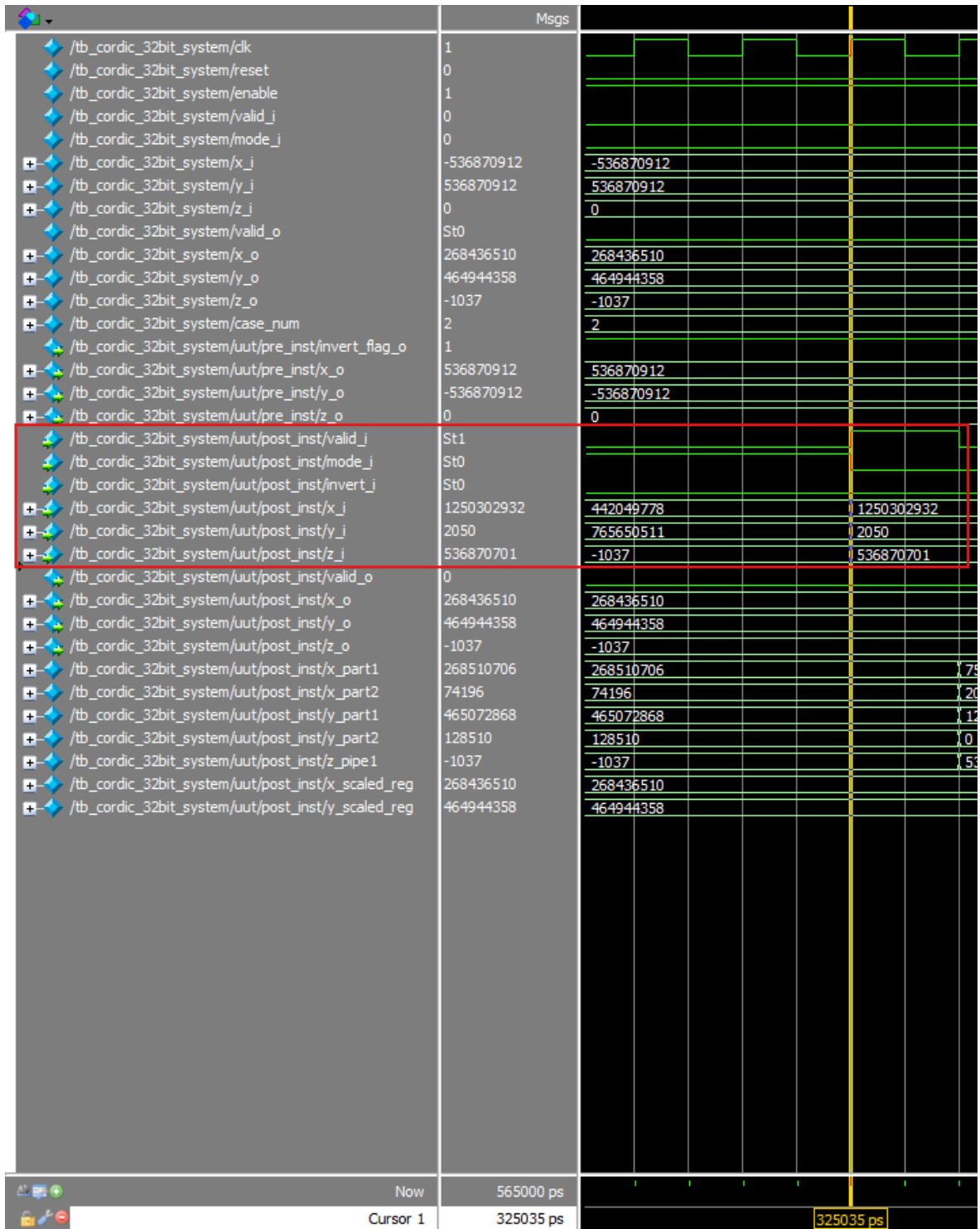
- INPUT:
 - $x = 1 \Rightarrow x \approx 536870912$ (Đổi $x = 1$ số thực sang định dạng Q2.29)
 - $y = 1 \Rightarrow y \approx 536870912$ (Đổi $y = 1$ số thực sang định dạng Q2.29)
 - $z = 0 \Rightarrow z \approx 0$ (Đổi $z = 0$ số thực sang định dạng góc chuẩn hóa).
 - $mode_i = 0$: Vectoring
 - $valid_i = 1$.



- Dữ liệu x, y, z, mode_i và valid_i đã sẵn sàng tại T = 120ns, lúc này dữ liệu đã đi qua khối tiền xử lý, vì test case 2 x = 1, y = 1, nó không nằm trong góc phần tư thứ hai hay thứ ba thì dữ liệu đi khối tiền xử lý

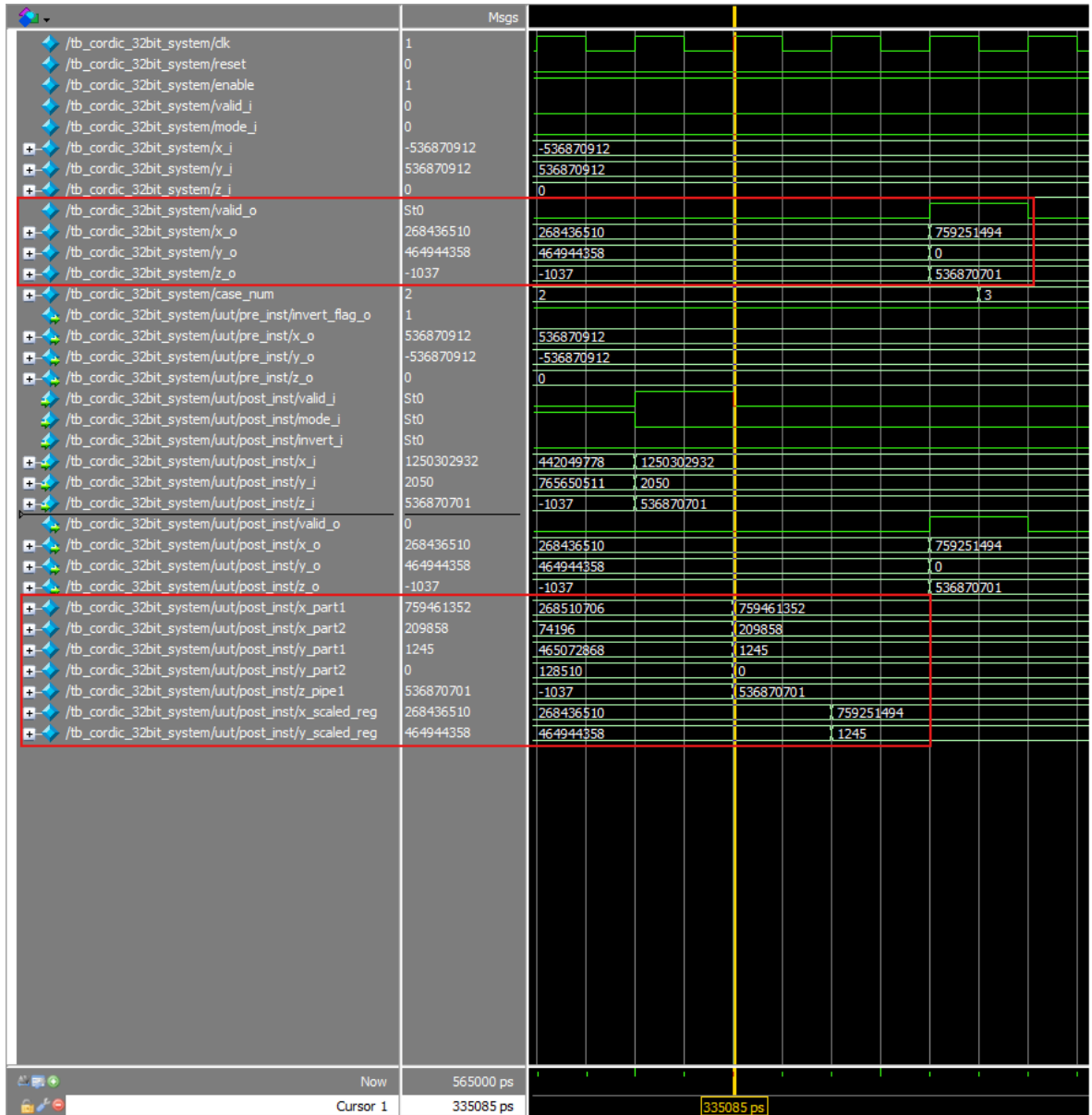
sẽ không thay đổi gì cả vì vậy cờ `invert_flag_o` cũng không được set lên 1.

- Sau khi dữ liệu đi qua khối tiền xử lý và đợi đến `clk` cạnh lên tiếp theo tại $T = 125\text{ns}$ nó sẽ được ghi vào thanh ghi đứng sau của khối tiền xử lý và đầu ra của nó được nối thẳng với `x_o`, `y_o`, `z_o` các tín hiệu này sẽ được nối vào tầng 0 của `DATAPATH_TOP` để tiến hành tính toán.



- Sau khi dữ liệu đi vào khối DATAPATH_TOP phải cần đến 20 chu kỳ xung clk sau thì ta mới nhận được output của DATAPATH_TOP cùng với lúc đó tín hiệu valid_i, mode_i, invert_i của phép tính này cũng được xuất ra từ khối CONTROLLER.

- Vì dữ liệu x y z chỉ mới được xử lý qua CORDIC_CORE nên có thể thấy x_o, y_o, z_o của khối DATAPATH_TOP lúc này là x_i, y_i, z_i của khối hậu xử lý.
- Vì đây là chế độ vectoring, x sẽ là độ lớn tổng hợp của vector, còn z sẽ là góc quay để đưa vector về trục hoành.
 - $x_i = 1250302932 \Rightarrow x \approx 2.32$ độ lớn tổng hợp của vector là bằng $\sqrt{x_i^2 + y_i^2} = \sqrt{1^2 + 1^2} = \sqrt{2} \approx 1.414$. Nhưng mà chúng ta có thể thấy độ dài của x đang bị dẫn ra gấp 1.64676 lần ($2.32 \approx 1.414 \times 1.64676$). Vì vậy chúng ta phải cần dữ liệu đi qua khối hậu xử lý để giải quyết việc đó.
 - $y_i = 2505 \approx 0.0000038$. Trong mode Vectoring, vì Y chắc chắn phải tiến về 0, nên con số 2050 này chỉ là sai số phần cứng. Thay vì đem sai số này đi nhân với hệ số bù, khối Post-processor của em đã chủ động ép nó về 0 luôn để làm sạch dữ liệu ngõ ra
 - $z_i = 536870701 \Rightarrow z \approx 44.999$ chính là giá trị của góc đã thực hiện quay (Vector có tọa độ x = 1, y = 1 thì góc của nó sẽ là 45 độ), vì đây là giá trị của góc nên nó không bị dẫn chúng ta không cần phải xử lý về độ dẫn giống như x và y.

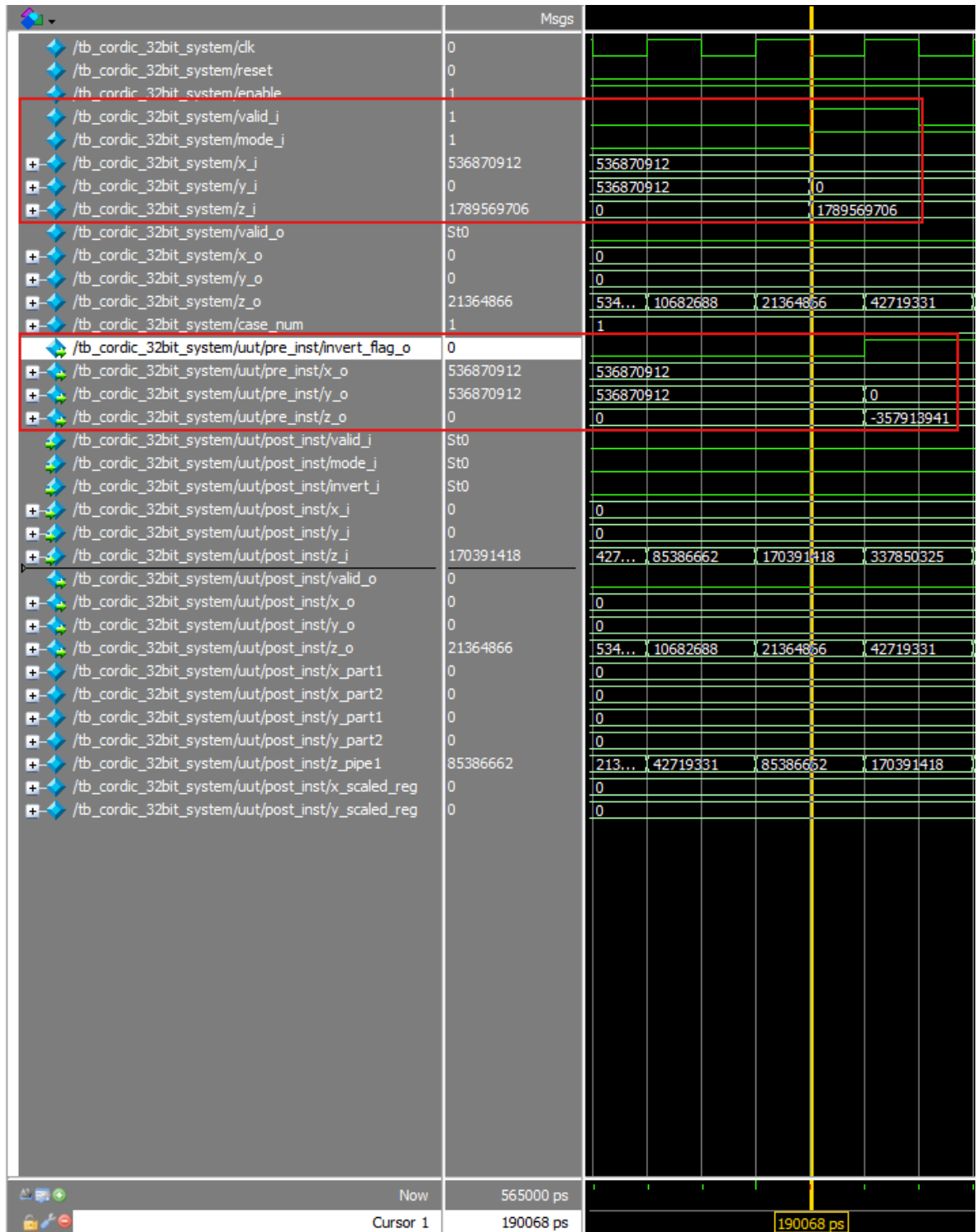


- Vì khối hậu xử lý thực hiện phép nhân được chia thành hai giai đoạn. Giai đoạn đầu tiên sẽ được xử lý và lưu vào x_part_1, x_part_2, y_part_1, y_part_2. Giai đoạn thứ hai sẽ hoàn thành xong việc đưa độ dẫn của x y về chiều dài ban đầu. Tuy nhiên với chế độ vectoring các phép tính toán và ghi vào thanh ghi y_part_1, y_part_2 sẽ không có ý nghĩa bởi vì y sẽ được gán bằng 0 cố định. Có thể thấy:

■ $x_scaled_reg = 759251494 \Rightarrow x \approx 1.414$ (đúng với kết quả

mong đợi $\sqrt{x_i^2 + y_i^2} = \sqrt{1^2 + 1^2} = \sqrt{2} \approx 1.414$.

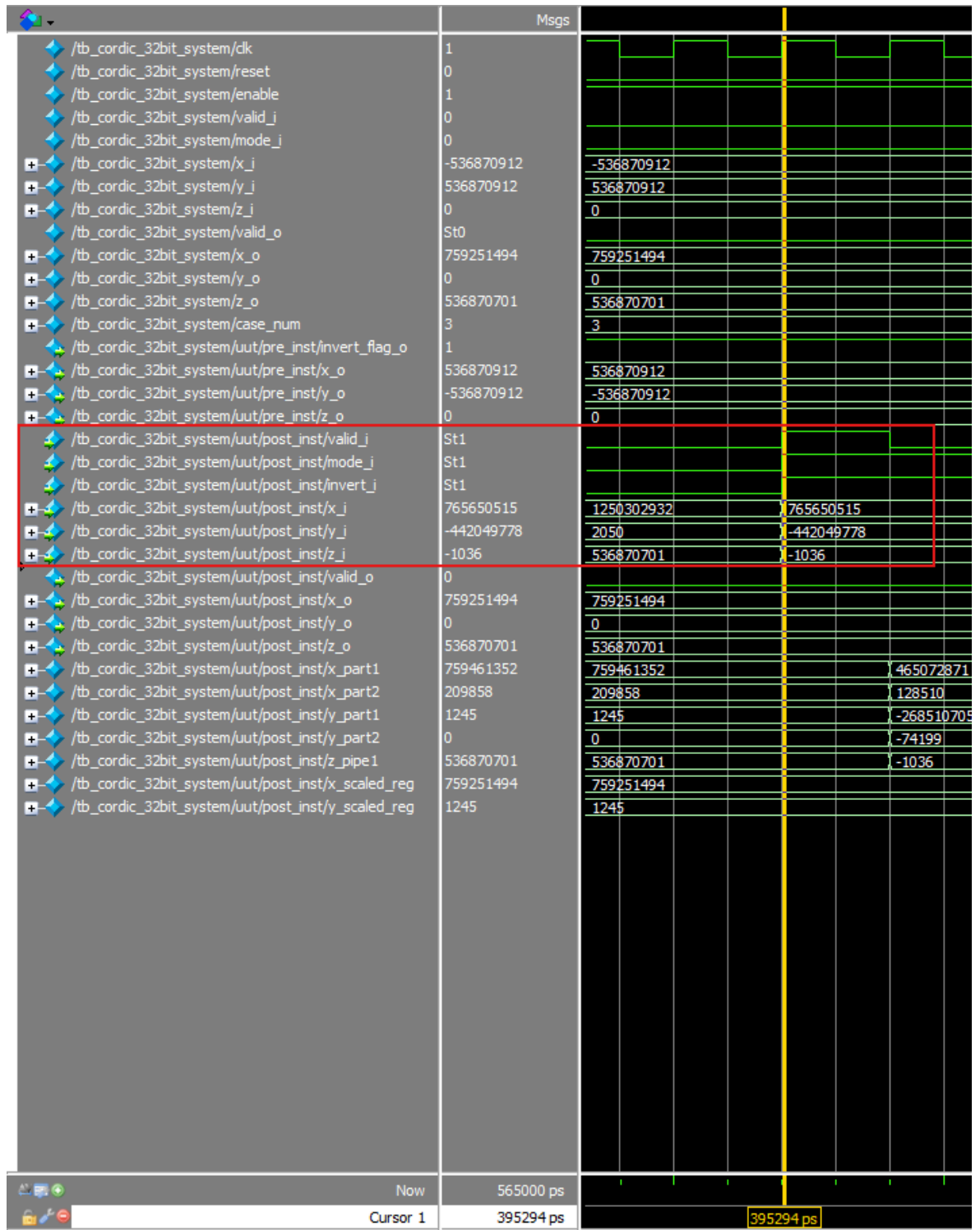
- $y_scaled_reg = 1245$ vì y sẽ được gán bằng 0 ở tầng cuối của khối hậu xử lý nên chúng ta có thể không cần quan tâm giá trị này.
 - Sau khi thực hiện triệt tiêu độ dẫn của x . Thì tầng tiếp theo là thực hiện ánh xạ lại. Nhưng mà do test case này cờ $invert = 0$ nên kết quả sẽ không thay đổi gì so với tầng thứ hai ngoài việc chúng ta sẽ ép y về 0. Kết quả đó sẽ được ghi vào thanh ghi cuối cùng và xuất ra ngoài đồng thời tín hiệu $valid_o$ được $set = 1$. Ngõ ra của khối hậu xử lý cũng là ngõ ra của cả mạch.
- **Test case 3**
 - INPUT:
 - $x = 1 \Rightarrow x \approx 536870912$ (Đổi $x = 1$ số thực sang định dạng Q2.29)
 - $y = 0 \Rightarrow y \approx 0$ (Đổi $y = 0$ số thực sang định dạng Q2.29)
 - $z = 150 \Rightarrow z \approx 1789569706$ (Đổi $z = 150$ số thực sang định dạng góc chuẩn hóa). Đây là trường hợp góc cần tính ở góc phần tư thứ 2.
 - $mode_i = 1$: Rotation
 - $valid_i = 1$.



- Dữ liệu x, y, z, mode_i và valid_i đã sẵn sàng tại T = 190ns, lúc này dữ liệu đã đi qua khối tiền xử lý, vì test case 3 z = 150, nó nằm trong

góc phần tư thứ hai thì dữ liệu sẽ được xử lý bởi khối tiền xử lý và cờ `invert_flag_o` sẽ được set lên 1.

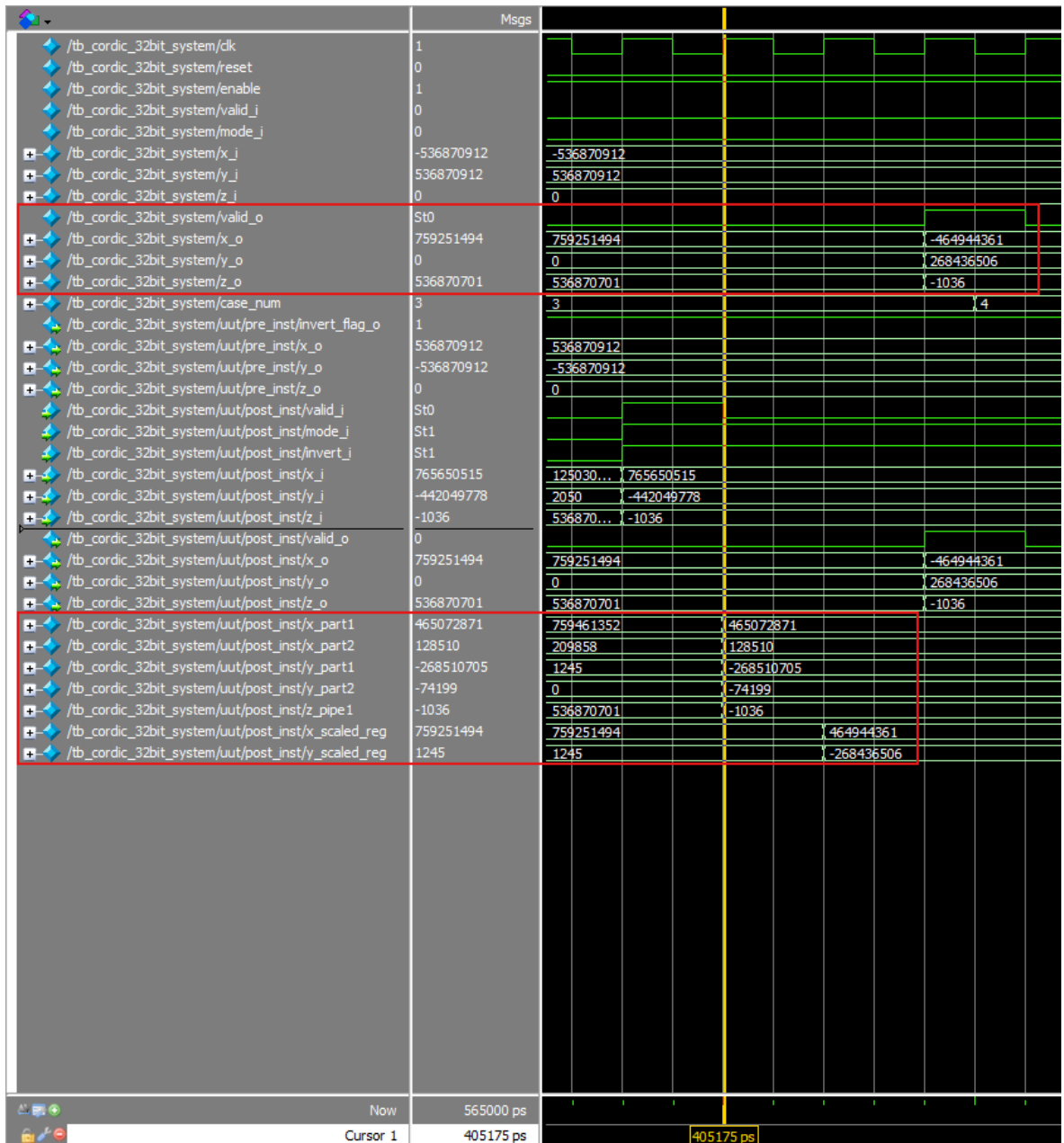
- Có thể thấy input `z` ban đầu là 1789569706 tương đương với $z = 150^\circ$ thì khi đi qua khối tiền xử lý thì nó đã xử lý và cho ra $z = -357913941$ tương đương với $z \approx -30^\circ$ (đưa về góc phần tư thứ 4).
- Sau khi dữ liệu đi qua khối tiền xử lý và đợi đến `clk` cạnh lên tiếp theo tại $T = 195\text{ns}$ nó sẽ được ghi vào thanh ghi đứng sau của khối tiền xử lý và đầu ra của nó được nối thẳng với `x_o`, `y_o`, `z_o` các tín hiệu này sẽ được nối vào tầng 0 của `DATAPATH_TOP` để tiến hành tính toán.



- Sau khi dữ liệu đi vào khối DATAPATH_TOP phải cần đến 20 chu kỳ xung clk sau thì ta mới nhận được output của DATAPATH_TOP cùng

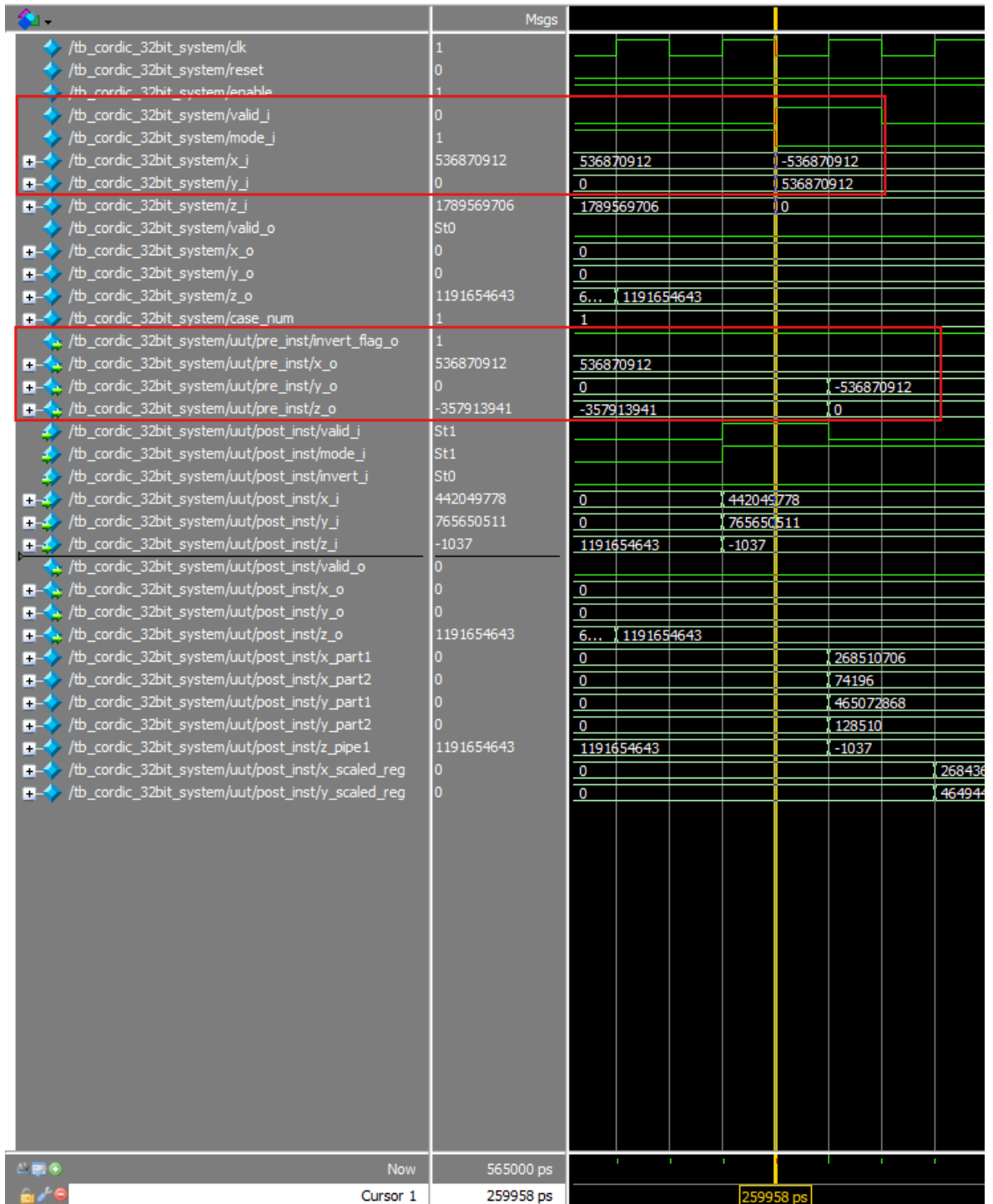
với lúc đó tín hiệu `valid_i`, `mode_i`, `invert_i` của phép tính này cũng được xuất ra từ khối CONTROLLER.

- Vì dữ liệu `x y z` chỉ mới được xử lý qua CORDIC_CORE nên có thể thấy `x_o`, `y_o`, `z_o` của khối DATAPATH_TOP lúc này là `x_i`, `y_i`, `z_i` của khối hậu xử lý:
 - $x_i = 765650515 \Rightarrow x \approx 1.426$ ($x = \cos(z) = \cos(-30) = 0.86$).
Nhưng mà chúng ta có thể thấy độ dài của `x` đang bị dẫn ra gấp 1.64676 lần ($1.426 \approx 0.86 \times 1.64676$). Vì vậy chúng ta phải cần dữ liệu đi qua khối hậu xử lý để giải quyết việc đó.
 - $y_i = -442049778 \Rightarrow y \approx -0.823$ ($y = \sin(z) = \sin(-30) = -0.5$).
Nhưng mà chúng ta có thể thấy độ dài của `y` đang bị dẫn ra gấp 1.64676 lần ($-0.823 \approx -0.5 \times 1.64676$). Vì vậy chúng ta phải cần dữ liệu đi qua khối hậu xử lý để giải quyết việc đó.
 - $z_i = -1037 \Rightarrow z \approx -0.0000869$ (gần về 0) nó là giá trị của góc nên không bị dẫn. Việc `z` trừ dần về 0 thì đây là cách hoạt động của CORDIC.



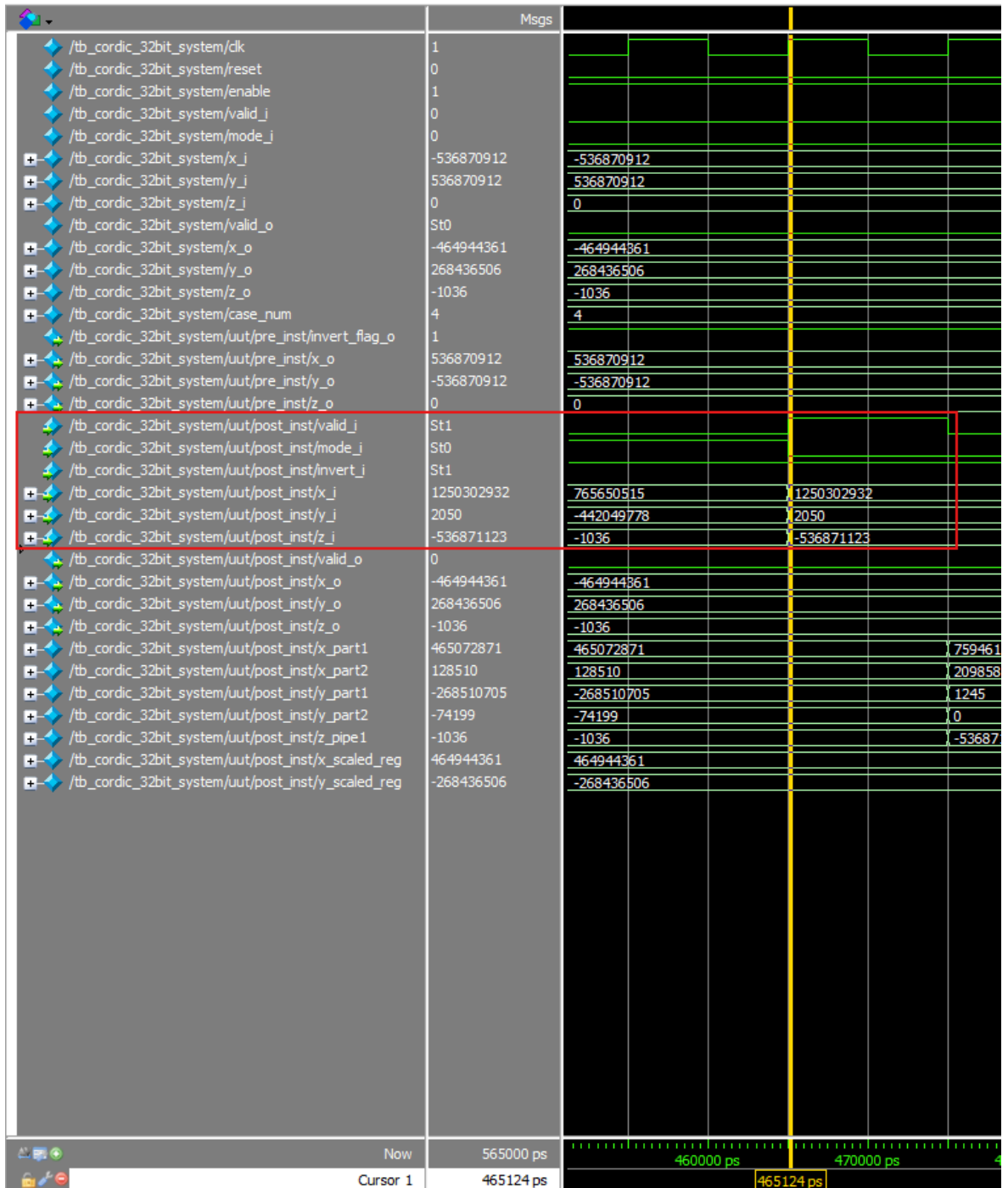
- Vì khối xử lý thực hiện phép nhân được chia thành hai giai đoạn. Giai đoạn đầu tiên sẽ được xử lý và lưu vào x_part_1, x_part_2, y_part_1, y_part_2. Giai đoạn thứ hai sẽ hoàn thành xong việc đưa độ dẫn của x y về chiều dài ban đầu. Có thể thấy:
 - $x_scaled_reg = 464944361 \Rightarrow x \approx 0.866$ (đúng với kết quả mong đợi $\cos(-30) = 0.866$).

- $y_scaled_reg = -268436506 \Rightarrow y \approx -0.5$ (đúng với kết quả mong đợi $\sin(-30) = -0.5$).
 - Sau khi chuyển chiều dài x y về chiều dài ban đầu. Thì tăng tiếp theo là thực hiện ánh xạ lại. Vì phép tính này cờ invert đang được set bằng 1 là do ta thực hiện chuyển z ban đầu từ 150 sang -30 nên ở tầng cuối của của khối hậu xử lý đã thực hiện đảo dấu lại từ $x_scaled_reg = 464944361$ sang -464944361 tương đương với $\cos(150) \approx -0.866$ và $y_scaled_reg = -268436506$ sang 268436506 tương đương với $\sin(150) \approx 0.5$.
 - Kết quả đó sẽ được ghi vào thanh ghi cuối cùng và xuất ra ngoài đồng thời tín hiệu `valid_o` được set = 1. Ngõ ra của khối hậu xử lý cũng là ngõ ra của cả mạch.
- **Test case 4**
 - INPUT:
 - $x = -1 \Rightarrow x \approx -536870912$ (Đổi $x = -1$ số thực sang định dạng Q2.29). Đây là trường hợp $x < 0$, vector đang nằm ở góc phần tư thứ 2.
 - $y = 1 \Rightarrow y \approx 536870912$ (Đổi $y = 1$ số thực sang định dạng Q2.29)
 - $z = 0 \Rightarrow z \approx 0$ (Đổi $z = 0$ số thực sang định dạng góc chuẩn hóa).
 - `mode_i = 0`: Vectoring
 - `valid_i = 1`.



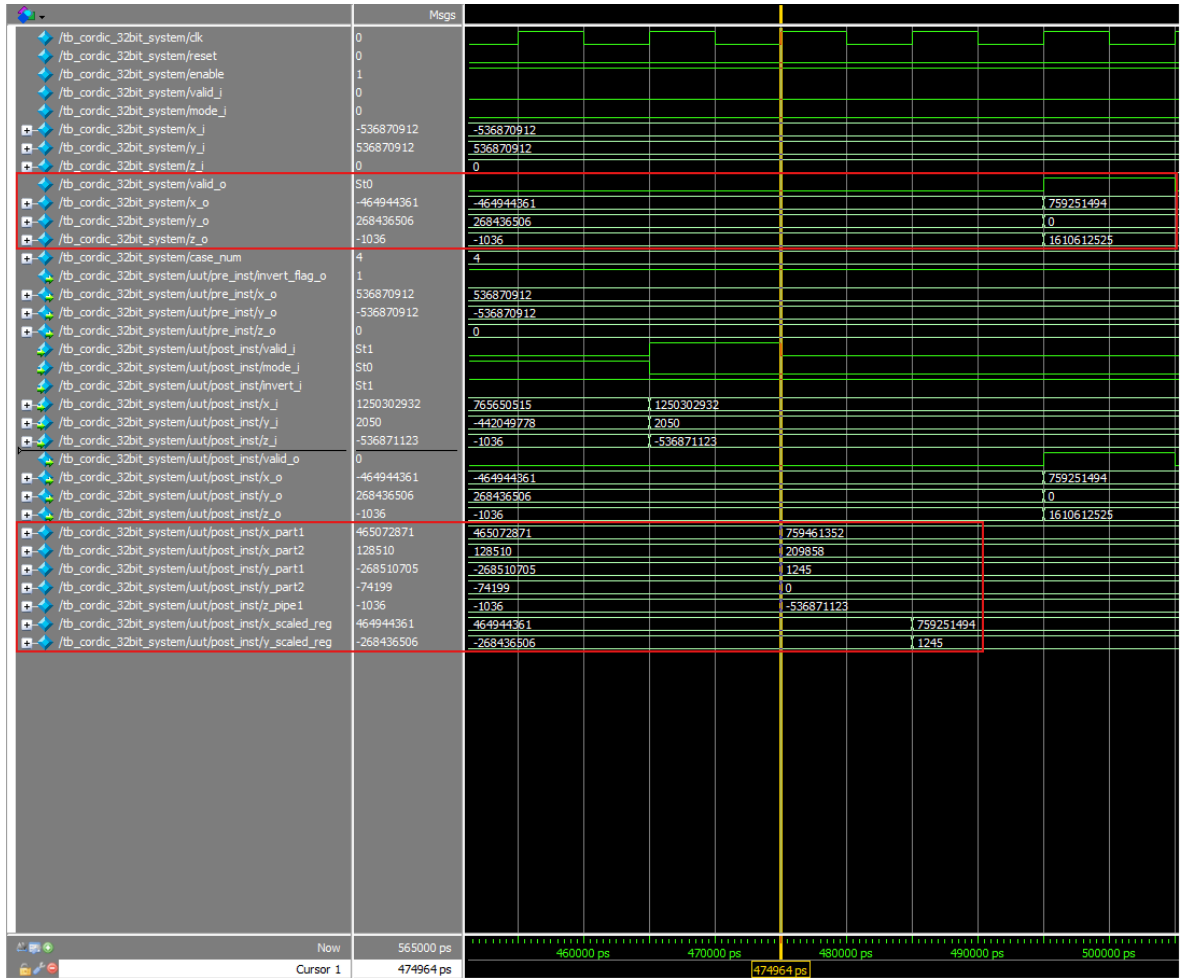
- Dữ liệu x, y, z, mode_i và valid_i đã sẵn sàng tại T = 260ns,, lúc này dữ liệu đã đi qua khối tiền xử lý, vì test case 4, x = -1, y = 1, nó nằm trong góc phần tư thứ hai thì dữ liệu sẽ được xử lý bởi khối tiền xử lý và cờ invert_flag_o sẽ được set lên 1.

- Có thể thấy ban đầu $x_i = -1$ thì sau khi đi qua khối tiền xử lý thì nó đã đảo lại thành 1 (quan sát ở x_o của pre) ≈ 536870912 . Và cũng thực hiện đảo luôn giá trị $y = 1$ ban đầu thành $y = -1$.
- Sau khi dữ liệu đi qua khối tiền xử lý và đợi đến clk cạnh lên tiếp theo tại $T = 265\text{ns}$ nó sẽ được ghi vào thanh ghi đứng sau của khối tiền xử lý và đầu ra của nó được nối thẳng với x_o, y_o, z_o các tín hiệu này sẽ được nối vào tầng 0 của DATAPATH_TOP để tiến hành tính toán.



- Sau khi dữ liệu đi vào khối DATAPATH_TOP phải cần đến 20 chu kỳ xung clk sau thì ta mới nhận được output của DATAPATH_TOP cùng với lúc đó tín hiệu valid_i, mode_i, invert_i của phép tính này cũng được xuất ra từ khối CONTROLLER.

- Vì dữ liệu x y z chỉ mới được xử lý qua CORDIC_CORE nên có thể thấy x_o, y_o, z_o của khối DATAPATH_TOP lúc này là x_i, y_i, z_i của khối hậu xử lý.
- Vì đây là chế độ vectoring, x sẽ là độ lớn tổng hợp của vector, còn z sẽ là góc quay để đưa vector về trục hoành.
 - $x_i = 1250302932 \Rightarrow x \approx 2.32$ độ lớn tổng hợp của vector là bằng $\sqrt{x_i^2 + y_i^2} = \sqrt{1^2 + (-1)^2} = \sqrt{2} \approx 1.414$. Nhưng mà chúng ta có thể thấy độ dài của x đang bị dẫn ra gấp 1.64676 lần ($2.32 \approx 1.414 \times 1.64676$). Vì vậy chúng ta phải cần dữ liệu đi qua khối hậu xử lý để giải quyết việc đó.
 - $y_i = 2505 \approx 0.0000038$. Trong mode Vectoring, vì Y chắc chắn phải tiến về 0, nên con số 2050 này chỉ là sai số phần cứng. Thay vì đem sai số này đi nhân với hệ số bù, khối Post-processor của em đã chủ động ép nó về 0 luôn để làm sạch dữ liệu ngõ ra
 - $z_i = -536870701 \Rightarrow z \approx -44.999$ chính là giá trị của góc đã thực hiện quay (Vector có tọa độ x = 1, y = -1 thì nó nằm ở góc -45 độ).



- Vì khối hậu xử lý thực hiện phép nhân được chia thành hai giai đoạn. Giai đoạn đầu tiên sẽ được xử lý và lưu vào x_part_1, x_part_2, y_part_1, y_part_2. Giai đoạn thứ hai sẽ hoàn thành xong việc đưa độ dẫn của x y về chiều dài ban đầu. Tuy nhiên với chế độ vectoring các phép tính toán và ghi vào thanh ghi y_part_1, y_part_2 sẽ không có ý nghĩa bởi vì y sẽ được gán bằng 0 cố định. Có thể thấy:

- $x_scaled_reg = 759251494 \Rightarrow x \approx 1.414$ (đúng với kết quả mong đợi

$$\sqrt{x_i^2 + y_i^2} = \sqrt{1^2 + (-1)^2} = \sqrt{2} \approx 1.414.$$

- $y_scaled_reg = 1245$ vì y sẽ được gán bằng 0 ở tầng cuối của khối hậu xử lý nên chúng ta có thể không cần quan tâm giá trị này.

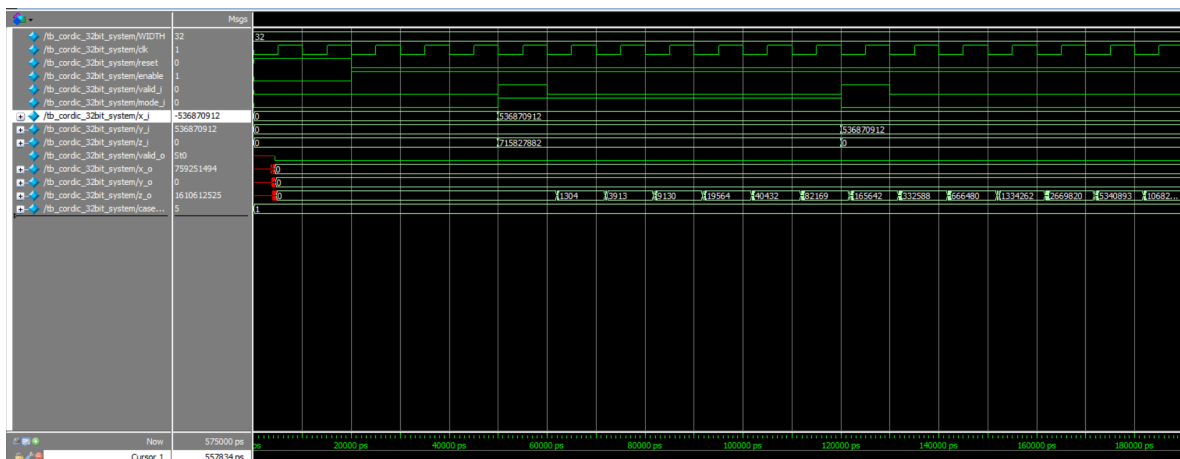
- Sau khi thực hiện triệt tiêu độ dẫn của x. Thì tăng tiếp theo là thực hiện ánh xạ lại. Chính vì sự kiện đảo từ $x = -1, y = 1$ sang $x = 1, y = -1$. Cờ invert đã được set lên 1 trước đó ta phải thực hiện cộng z với 180 và ta sẽ được $z = -45 + 180 = 135 \approx 161610612525$.
- Dữ liệu sẽ được chốt vào thanh ghi cuối và xuất ra ngoài cùng với lúc đó thì cờ valid_o cũng được bật lên 1.

4.3. Đánh giá kết quả mô phỏng mức cổng vật lý (Post-simulation)

4.3.1. Mục đích và môi trường kiểm chứng

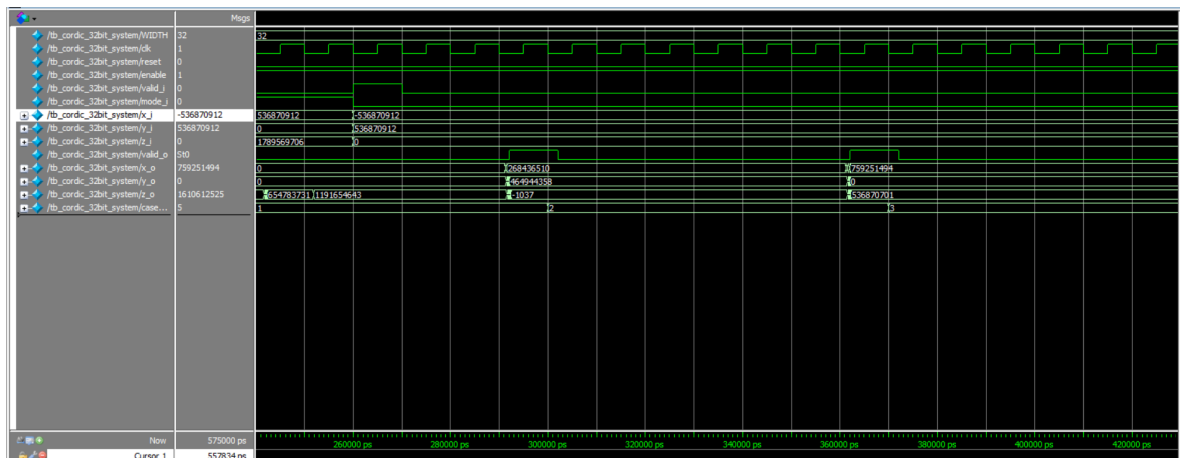
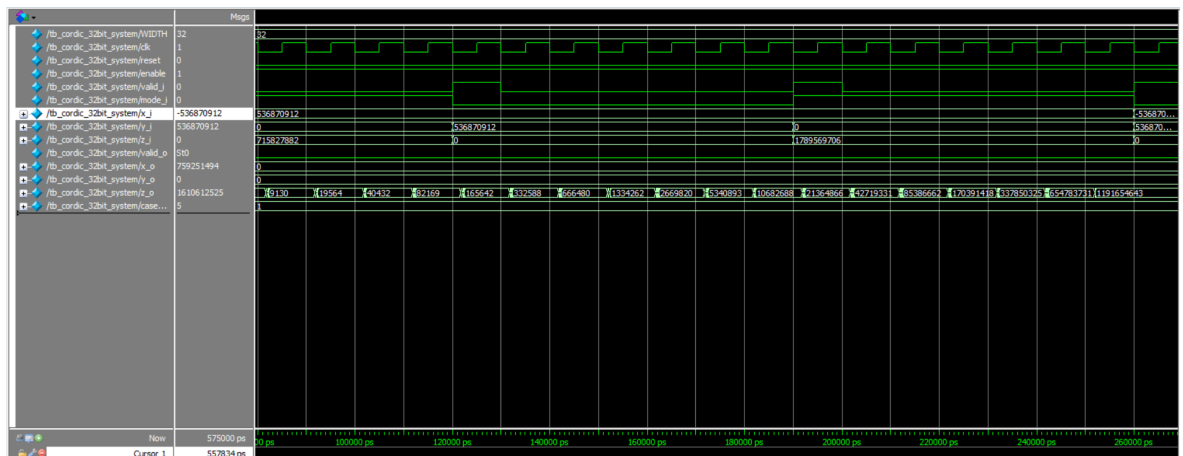
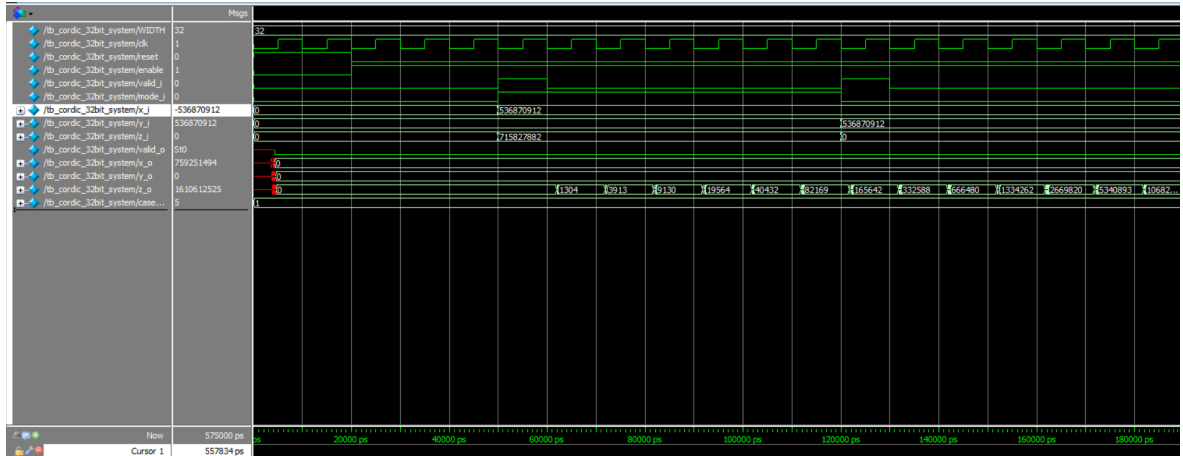
Khác với mô phỏng Pre-simulation / RTL, mô phỏng mức cổng vật lý (Post-simulation) trên ModelSim yêu cầu hệ thống phải biên dịch cùng với tệp tin thể vật lý chuẩn (file .sdo) được trích xuất từ phần mềm Quartus sau quá trình tổng hợp Synthesis. Quá trình này mô phỏng sát nhất với thực tế hiện tượng trễ qua cổng logic (Gate Delay) và trễ do đi dây (Routing Delay) trên bề mặt chip silic.

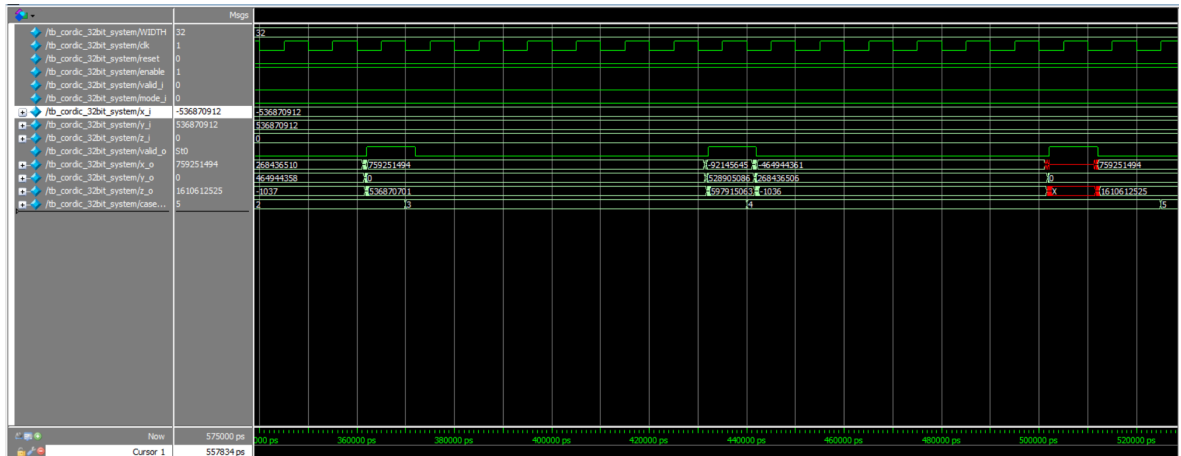
4.3.2. Đánh giá độ trễ đường ống



nhỏ để sạc/xả tụ điện bên trong thì ngõ ra Q mới đảo trạng thái được nên sẽ sinh ra độ trễ lan truyền này.

4.3.3. Đánh giá kết quả cho các testcases





- Test case 1:** chế độ Rotation với giá trị nạp vào $mode_i = 1$, $X_{in} = 1.0 = 32'd536870912$, $Y_{in} = 0$ và góc xoay $Z_{in} = 60^\circ = 32'd715827882$
 - Thời điểm 50000 ps, cờ $valid_i$ bật lên 1, ta nạp các giá trị của testcase 1 vào.
 - Trải qua các tầng pipeline, sau đó ở thời điểm 302000 ps ngay tại sườn xuống của xung clock được viết trong testbench, **valid_o** sẽ bật lên 1, chốt cho các giá trị ngõ ra. X_o có giá trị 268436510 -> Quy đổi ra số thực $2268436510 / 229 \approx 0.500001$ -> Xấp xỉ với giá trị lý thuyết $\cos(60^\circ) = 0.5$.
 - Tương tự với Y_o có giá trị 464944358 -> Quy đổi ra số thực ≈ 0.866011 -> Xấp xỉ với giá trị lý thuyết $\sin(60^\circ) = 0.866025$.
 - Góc dư $Z_o = -1307 \approx 0.000086^\circ$ -> Xấp xỉ hội tụ về 0.
- Test case 2:** chế độ Vectoring với giá trị nạp vào $mode_i = 0$, $X_{in} = 1.0 = 32'd536870912$, $Y_{in} = 1.0 = 32'd536870912$ và $Z_{in} = 0$
 - Do kiến trúc pipeline nên ta sẽ nạp ngõ vào cho test case 2 khi các tầng đầu đã thực hiện xong phép tính toán cho test case 1.
 - Tại thời điểm 372000 ps, kết quả ngõ ra được ghi nhận. $X_o = 759251494 \approx 1.41421_{10}$ -> Xấp xỉ giá trị lý thuyết $\sqrt{1^2 + 1^2} = \sqrt{2}$
 - $Z_o = 536870701$ -> Quy đổi về số thực $(536870701 / 2^{31}) * 180 \approx 44.91^\circ$ -> Giá trị lý thuyết 45° .

- Y_o sẽ hội tụ về 0.
- **Test case 3:** chế độ Rotation với giá trị nạp vào $mode_i = 1$, $X_{in} = 1.0$, $Y_{in} = 0$, $Z_{in} = 1789569706$ (150°)
 - Tại thời điểm 442000 ps, kết quả thu được là $X_o = -464944361 \approx -0.866011 \rightarrow$ Xấp xỉ giá trị lý thuyết $\cos(150^\circ)$
 - $Y_o = 268436506 \approx 0.500001 \rightarrow$ Xấp xỉ giá trị lý thuyết của $\sin(150^\circ)$
 - Ta thấy có các giá trị X_o , Y_o khác trước khi cập nhật sang giá trị chính thức, đây là các dữ liệu từ chu kì trước do lỗi CORDIC tính toán liên tục. Sau khi có cạnh xuống của xung clock và đợi thêm một khoảng thời gian trễ, ngõ ra sẽ được cập nhật bằng giá trị chính thức.
- **Test case 4:** chế độ Vectoring với các giá trị nạp vào $mode_i = 1$, $X_{in} = -1.0$ (-536870912), $Y_{in} = 1.0$ (536870912)
 - Tại thời điểm 512000 ps, các giá trị sẽ được chốt. $X_o = 759251494 \rightarrow 1.41421$
 - $Z_o = 1610612525 \rightarrow (1610612525 / 2^{31}) * 180 \approx 134.99^\circ \rightarrow$ Xấp xỉ giá trị lý thuyết 135° .
 - Giá trị “x” mà chúng ta thấy ở mốc 500000 ps đó là việc gói dữ liệu ở Test case 4 đã ghi đè lên ngõ ra ở dữ liệu Test case 3 từ trước nhưng các thanh ghi Flip flop khiến cho có độ trễ lan truyền tạo nên. Sau đó khoảng 12000 ps nữa ngõ ra sẽ được cập nhật vào thanh ghi và cho ra giá trị chính xác cuối cùng.

4.4. Đánh giá thông số phần cứng

4.4.1. Tài nguyên sử dụng (Logic Elements, Registers)

Total logic elements	3,546 / 33,216 (11 %)
Total combinational functions	3,489 / 33,216 (11 %)
Dedicated logic registers	2,295 / 33,216 (7 %)
Total registers	2295
Total pins	198 / 475 (42 %)
Total virtual pins	0
Total memory bits	132 / 483,840 (< 1 %)
Embedded Multiplier 9-bit elements	0 / 70 (0 %)
Total PLLs	0 / 4 (0 %)

Loại tài nguyên	Số lượng sử dụng	Tổng phần trăm (%)	Chức năng đảm nhận trong lõi CORDIC
Logic Elements (LEs)	3.546	11 %	Xây dựng bộ cộng/trừ 32 bit, mạch dịch bit.
Dedicated Logic Registers (FFs)	2.295	7 %	Số lượng FFs chốt dữ liệu cho 24 tầng pipeline
Total Pins (I/O)	198/475	42 %	Giao tiếp các bus dữ liệu 32 bit (x_i , y_i , z_i , x_o , y_o , z_o) và tín hiệu điều khiển.

- Mức chiếm dụng 11% của phần tử logic tổ hợp tổng hợp thành các bộ cộng/trừ, dịch bit, cho thấy lõi CORDIC của nhóm khá nhỏ gọn, chừa 89% diện tích bề mặt chip còn lại cho các module khác.
- Theo tính toán lý thuyết, để trải qua 24 tầng pipeline của nhóm thì cần khoảng 3 flip flop với độ rộng 32 bit cho các biến X, Y, Z. Vì thế sẽ cần $3 * 32 * 24 = 2304$ Flipflops. Do đó lượng 2.295 FFs mà chúng ta đã sử dụng

gần xấp xỉ so với số liệu tính toán lý thuyết do có sự lược bỏ một vài FFs không trạng thái từ Quartus.

- Con số 198 chân I/O này phù hợp với kích thước cổng vật lý của chúng ta, với 6 bus dữ liệu 32 bit: $6 * 32 = 192$ chân tín hiệu, cộng với 6 chân I/O còn lại của các tín hiệu điều khiển clk, enable, reset, valid_i, mode_i, valid_o.

4.4.2. Tần số hoạt động tối đa (F_{max}) và Độ trễ (Latency)

4.4.2.1. Tần số hoạt động tối đa (F_{max})

Slow Model Fmax Summary				
	Fmax	Restricted Fmax	Clock Name	Note
1	128.12 MHz	128.12 MHz	clk	

Mức tần số trên đã đạt được so với yêu cầu > 100 MHz của đề tài. Tương ứng với chu kỳ xung nhịp mà mạch có thể đáp ứng an toàn là $T_{min} \approx 7.805$ ns.

4.4.2.2. Độ trễ (Lantency)

Có thể thấy ở thời điểm nhập các giá trị ngõ vào và khi có output sẵn sàng là từ 40000 - 28000 ps, tức là hệ thống chạy 240000 ps $\rightarrow$ ta có được 24 chu kỳ với mỗi chu kỳ 10000 ps.

Do đó tổng độ trễ hệ thống của nhóm là 24 chu kỳ xung nhịp, bao gồm: 1 chu kỳ từ khối Tiền xử lý (Preprocessor), 20 chu kỳ stages cho việc tính toán lõi CORDIC, 3 chu kỳ cho khối Hậu xử lý (Postprocessor).

4.4.3. Phân tích sai số

Do bản chất của CORDIC là một thuật toán xấp xỉ tiệm cận, nên kết quả của ngõ ra lõi CORDIC 32 bit không thể đạt được độ chính xác tuyệt đối như trên máy tính đa dụng được.

Hơn nữa, thuật toán lý thuyết cần vô hạn vòng lặp để vector hội tụ tuyệt đối. Tuy nhiên, do giới hạn của đề tài với tối đa 20 vòng lặp. Bước nhảy góc nhỏ nhất ở vòng lặp cuối cùng ($i=19$) là $\arctan(2^{-19}) \approx 0.000109^\circ$, nên ngõ ra z_0 không thể nào bằng 0

tuyệt đối được mà tồn tại một lượng sai số nhỏ dao động khoảng $\pm 0.000109^\circ$ quanh bước nhảy này.

Việc xử lý xấp xỉ hệ số K của khối Hậu xử lý cũng đã có một sai số nhỏ rồi với sự chênh lệch của kết quả chuỗi so với giá trị K lý thuyết đó là $\Delta K \approx 0.607252935 - 0.607254 \approx 0.000001$.

Ngoài ra, tại mỗi tầng của 20 vòng lặp, dữ liệu được dịch phải theo công thức thuật toán. Khi dịch bit ở mỗi tầng, các bit ở trọng số nhỏ nhất (LSB) sẽ bị đẩy ra khỏi thanh ghi 32 bit, cứ như thế liên tục qua 20 tầng sẽ gây ra sự sai lệch tích lũy vào kết quả cuối cùng.

Kết quả từ testcases:

	Giá trị tính được	Giá trị lý thuyết	Sai số
Rotation	$\cos(60^\circ) \approx 0.500001$ $\sin(60^\circ) \approx 0.866011$	$\cos(60^\circ) = 0.5$ $\sin(60^\circ) = 0.866025$	$\Delta X = 10^{-6}$ $\Delta Y = 1.4 * 10^{-5}$
Vectoring	$z = 44.99^\circ$	$z = 45^\circ$	$\Delta Z = 0.01^\circ$

Các kết quả từ testcases cũng cho ra kết quả sai số dao động trong khoảng 10^{-5} và sai số góc không vượt quá 0.01° .

Chương 5. KẾT LUẬN

5.1. Kết quả đạt được

Sau một thời gian tập trung nghiên cứu, thiết kế và thực nghiệm, đề án "**Thiết kế và tối ưu hóa lõi IP CORDIC 32-bit trên FPGA**" đã hoàn thành trọn vẹn các mục tiêu ban đầu:

- Hiểu được nguyên lý nền tảng của thuật toán CORDIC ở hai chế độ Rotation và Vectoring.
- Thiết kế thành công lõi CORDIC 32 bit bằng ngôn ngữ verilog HDL với cấu trúc pipeline gồm 24 tầng với 20 vòng lặp cho việc tính toán CORDIC, 4 tầng còn lại cho khâu xử lý input/ output cho khối Tiền xử lý và Hậu xử lý.
- Tốc độ xử lý $F_{\max} = 128.12$ MHz đạt yêu cầu của đề tài > 100 MHz, chỉ sử dụng các bộ cộng/ trừ và dịch bit mà không cần sử dụng bộ nhân nào.
- Xây dựng được môi trường kiểm thử Testbench thông qua 4 testcases cho cả Pre-simulation và Post-simulation.

5.2. Những điểm hạn chế của đề tài

- Việc sử dụng chuẩn số phẩy tĩnh Q2.29 cho tọa độ và Q0.31 cho góc bị thu hẹp đáng kể so với chuẩn số phẩy động. Mạch sẽ hoạt động kém hiệu quả hoặc xảy ra hiện tượng tràn số học (Overflow) nếu ngõ vào là quá lớn hoặc quá nhỏ sát ngưỡng 0. Để sử dụng hệ thống ta phải chuẩn hóa đầu vào về khoảng $[-1.0 ; 1.0]$.
- Do độ trễ của hệ thống được cố định vào mức 24 chu kì, đối với những góc đầu vào là $Z = 0^\circ$ không cần xoay hay vector hội tụ từ rất sớm ở các vòng lặp đầu thì vẫn phải đi qua hết 24 chu kì mới được chốt dữ liệu ngõ ra. Điều này gây ra lãng phí năng lượng chuyển mạch tại các tầng không tính toán giá trị.

- Hiện tại hệ thống được đóng gói ở dạng các cổng tín hiệu thô với các cờ báo cơ bản, thuận tiện cho việc mô phỏng kiểm chứng trên Modelsim/ Quartus nhưng chưa thể tích hợp cho các hệ thống SoC hiện đại được.